

ALAMO Adsorption

In []: %reset

```
In [1]: from idaes.core.surrogate.alamopy import AlamoTrainer, AlamoSurrogate
from idaes.core.surrogate.plotting.sm_plotter import surrogate_scatter2D, surrogate_parity, surrogate_residual
import time
import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import r2_score, mean_squared_error, max_error

df= pd.read_csv('train_data_winsorized.csv')
#print(df)
dataset = np.array(df)

#Scale the data between 0 and 1
scaler = MinMaxScaler((0,1))
print(scaler.fit(dataset))
scaled_dataset = scaler.transform(dataset)
#print(scaled_dataset)

#Inputs, X1 ..... X8 (matrix X)
x = scaled_dataset[:,0:5]

#Outputs
y1 = scaled_dataset[:,5]  #y1 - Adsorption Rate

feature_names = ['T', 'Humidity', 'AdsTime', 'DesTime', 'SteamVelocity']
output_names = ['q_ads'] #, 'q_des[mol/kg]'

df_scaled = pd.DataFrame({'T': scaled_dataset[:, 0], 'Humidity': scaled_dataset[:, 1],
                          'AdsTime':scaled_dataset[:, 2] , 'DesTime':scaled_dataset[:, 3],
                          'SteamVelocity':scaled_dataset[:, 4],
                          'q_ads':scaled_dataset[:, 5]})

dfin_scaled = df_scaled[feature_names]
dfout_scaled = df_scaled[output_names]

print(df_scaled)
```

```
MinMaxScaler()  
      T Humidity  AdsTime  DesTime  SteamVelocity  q_ads  
0      0.653167  1.000000  0.556049  0.697732      0.045917  0.901366  
1      0.110306  1.000000  1.000000  0.218832      0.427028  0.999326  
2      0.940640  0.427037  0.556049  0.382874      0.456776  0.664713  
3      0.000000  1.000000  0.445061  0.203217      0.846560  0.551573  
4      0.339751  1.000000  0.557159  0.000000      0.444444  0.625847  
...      ...      ...      ...      ...      ...  
16533  0.623476  0.553551  0.556049  0.029486      0.451578  0.969987  
16534  0.110306  1.000000  0.112098  0.252773      0.827541  0.296413  
16535  0.634202  0.664214  0.112098  0.360672      0.764549  0.100387  
16536  0.940640  1.000000  0.778024  0.122920      1.000000  0.850760  
16537  0.000000  1.000000  0.223085  0.508044      0.693120  0.290176  
  
[16538 rows x 6 columns]
```

```
In [2]: trainer = AlamoTrainer(input_labels= dfin_scaled,
                             output_labels= dfout_scaled,
                             training_dataframe= df_scaled)
trainer.config.constant = True
trainer.config.linfcns = True
trainer.config.monomialpower = [2, 3]
trainer.config.multi2power = [1, 2]

# Train surrogate (calls ALAMO through IDAES ALAMOPy wrapper)
from time import time
t0 = time()
success, alm_surr, msg = trainer.train_surrogate()
training_time = time() - t0
print(f"Training time = {(training_time)}s")

# save model to JSON
model = alm_surr.save_to_file(f'alamo_pce3_surrogate.json', overwrite=True)

# create callable surrogate object
surrogate_expressions = trainer._results['Model']

input_labels = trainer._input_labels
output_labels = trainer._output_labels
#xmin, xmax = [0.1, 0.8], [0.8, 1.2]
#input_bounds = {input_labels[i]: (xmin[i], xmax[i])
                 #for i in range(len(input_labels))}

alm_surr = AlamoSurrogate(surrogate_expressions,
                          input_labels,
                          output_labels)
                          #input_bounds)

surrogate_parity(alm_surr, df_scaled, filename= f'train_parity_alamo_pce3.pdf')

df_predictions_train =alm_surr.evaluate_surrogate(dfin_scaled)
# Append '_pred' to each column name
df_predictions_train.rename(columns=lambda x: x + '_pred', inplace=True)
print(df_predictions_train)

#Put the testing output data next to the predicted data
data_train= pd.concat([dfout_scaled, df_predictions_train], axis=1)
print(data_train)
```

```

#Performance metrics on the test set
r2_score_train = r2_score(dfout_scaled, df_predictions_train)
print(f"R^2 on training data : {r2_score_train}")

mse = mean_squared_error(dfout_scaled, df_predictions_train)
print('MSE=',mse)
rmse = np.sqrt(mse)
print('RMSE=', rmse)

max_error_train= max_error(dfout_scaled, df_predictions_train)
print('Maximum Absolute Error=', max_error_train)

#Parity plot
z=[0,1]
w=[0,1]
fig, ax= plt.subplots(figsize=(6, 5))
ax.scatter(dfout_scaled, df_predictions_train, alpha=0.3)
ax.plot(z, w, 'k--') #y=x line
ax.set_xlabel('Actual', fontsize=12)
ax.set_ylabel('Predicted', fontsize=12)
ax.set_title('q_ads', fontsize=10)

plt.tight_layout()
plt.savefig(f'alamo_parity_train_q_ads.png', format='png') # Save plot as a PNG file
plt.show()
plt.close(fig) # Close the figure to free memory

```

ALAMO version 2023.2.13. Built: WIN-64 Mon Feb 13 21:30:56 EST 2023

If you use this software, please cite:
 Cozad, A., N. V. Sahinidis and D. C. Miller,
 Automatic Learning of Algebraic Models for Optimization,
 AIChE Journal, 60, 2211-2227, 2014.

ALAMO is powered by the BARON software from <http://www.minlp.com/>

Licensee: Dimitrios Fardis at Georgia Institute of Technology, dfardis3@gatech.edu.

Reading input data
 Checking input consistency and initializing data structures

Step 0: Initializing data set

User provided an initial data set of 16538 data points

We will sample no more data points at this stage

Iteration 1 (Approx. elapsed time 0.88 s)

Step 1: Model building using BIC

Model building for variable q_ads

BIC = -0.564E+05 with q_ads = 1.2 * Adstime

BIC = -0.645E+05 with q_ads = 1.9 * Adstime - 0.91 * Adstime^2

BIC = -0.664E+05 with q_ads = 2.0 * Adstime - 1.0 * Adstime^2 - 0.27 * DesTime*SteamVelocity

BIC = -0.757E+05 with q_ads = 2.3 * T + 3.8 * Humidity + 0.43 * Adstime - 0.76 * DesTime - 1.4 * SteamVelocity - 1.2 * T^2 - 3.8 * Humidity^2 + 0.16 * Adstime^2 + 0.98 * DesTime^2 + 1.9 * SteamVelocity^2 + 0.22E-01 * T^3 + 1.5 * Humidity^3 - 0.46 * Adstime^3 - 0.61 * DesTime^3 - 0.95 * SteamVelocity^3 - 2.6 * T*Humidity + 0.38 * T*Adstime - 0.29 * T*DesTime - 0.12 * T*SteamVelocity - 0.90E-01 * Humidity*Adstime - 0.24 * Humidity*DesTime - 0.31 * Humidity*SteamVelocity + 0.65 * Adstime*DesTime + 0.96 * Adstime*SteamVelocity - 0.10 * DesTime*SteamVelocity + 1.3 * (T*Humidity)^2 - 0.21 * (T*Adstime)^2 + 0.12 * (T*DesTime)^2 - 0.40E-01 * (T*SteamVelocity)^2 + 0.18 * (Humidity*Adstime)^2 + 0.23 * (Humidity*DesTime)^2 + 0.18 * (Humidity*SteamVelocity)^2 - 0.17 * (Adstime*DesTime)^2 - 0.43 * (Adstime*SteamVelocity)^2 - 0.61

Calculating quality metrics on observed data set.

Quality metrics for output q_ads

SSE OLR:	166.
SSE:	166.
RMSE:	0.100
R2:	0.888
R2 adjusted:	0.888
Model size:	35
BIC:	-0.757E+05
Cp:	-0.163E+05
AICc:	-0.760E+05
HQC:	-0.759E+05
MSE:	0.101E-01
SSEp:	166.
RIC:	417.
MADp:	0.216E+04

Total execution time 16. s

Times breakdown

OLR time: 12. s in 7853 ordinary linear regression problem(s)

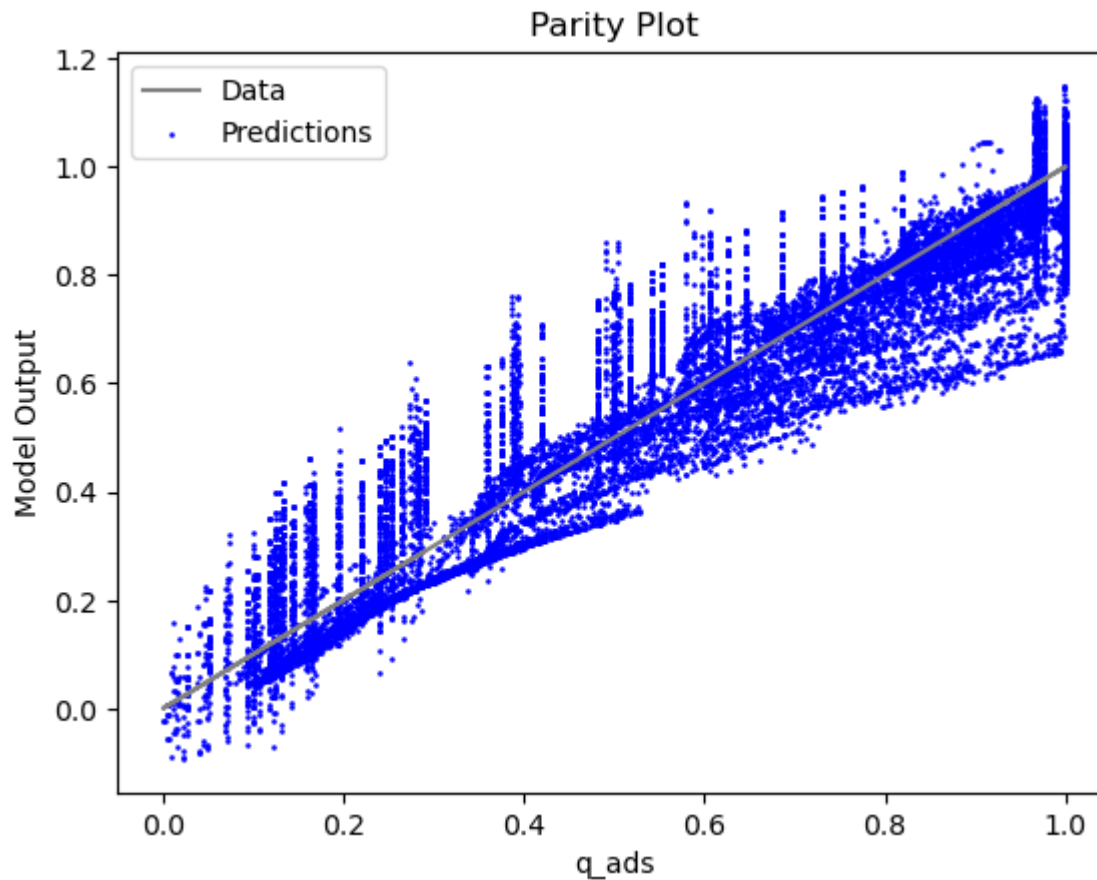
MINLP time: 0.0 s in 0 optimization problem(s)

Simulation time: 0.0 s to simulate 0 point(s)

All other time: 4.3 s in 1 iteration(s)

Normal termination

Training time = 17.724815845489502s



	q_ads_pred
0	0.840326
1	0.943972
2	0.681627
3	0.698301
4	0.837139
...	...
16533	0.897870
16534	0.242261
16535	0.127279
16536	0.872538
16537	0.350860

[16538 rows x 1 columns]

	q_ads	q_ads_pred
0	0.901366	0.840326
1	0.999326	0.943972
2	0.664713	0.681627
3	0.551573	0.698301
4	0.625847	0.837139
...	...	...
16533	0.969987	0.897870
16534	0.296413	0.242261
16535	0.100387	0.127279
16536	0.850760	0.872538
16537	0.290176	0.350860

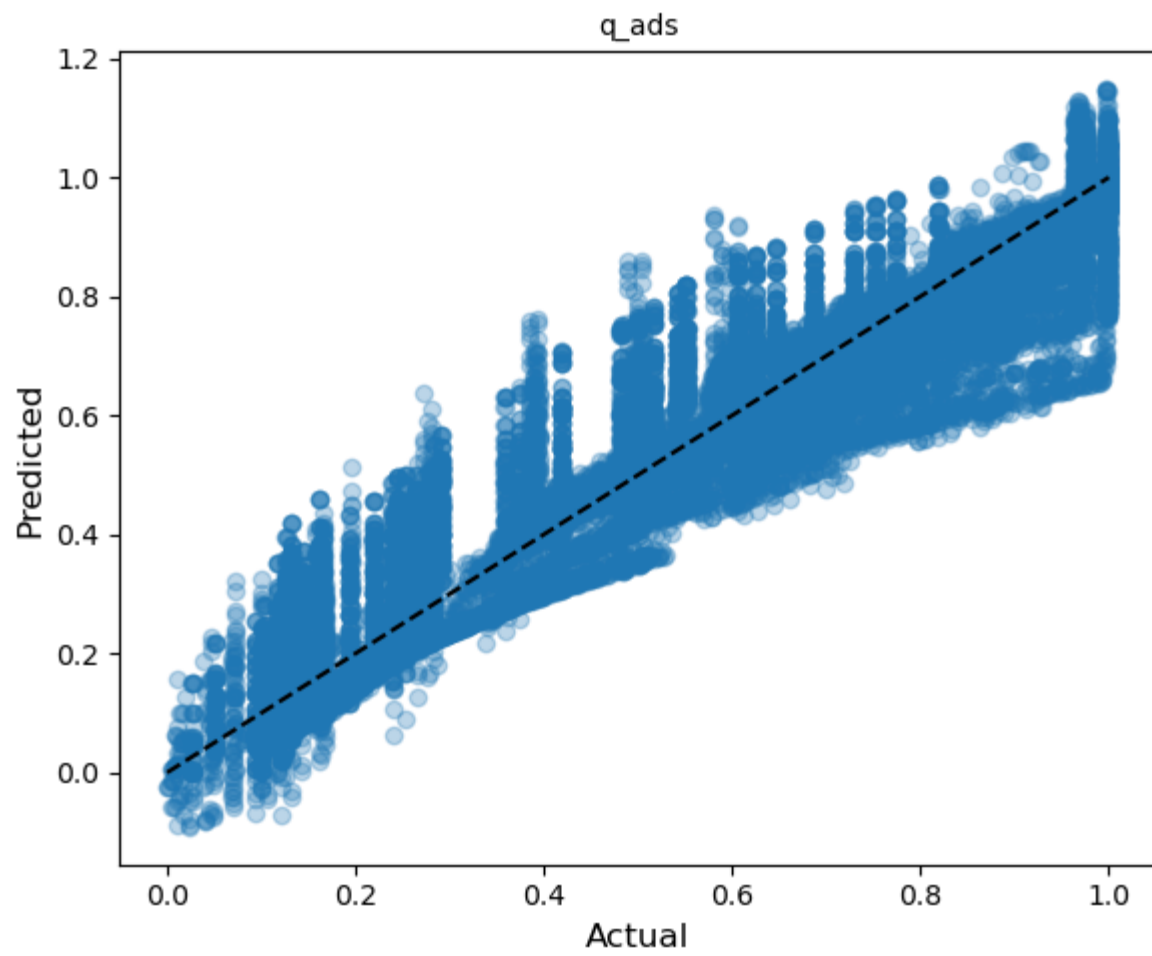
[16538 rows x 2 columns]

R^2 on training data : 0.8883682616469696

MSE= 0.010046581865373279

RMSE= 0.10023263872298922

Maximum Absolute Error= 0.37356314004867175



```

In [3]: #-----Testing set-----
df_test= pd.read_csv('test_data_unprocessed.csv')
print(df_test)
dataset_test = np.array(df_test)

#Scale the data between 0 and 1 using the scaler based on the training set
scaled_dataset_test = scaler.transform(dataset_test)
#print(scaled_dataset_test)

#Inputs, X1 ..... X8 (matrix X)
x_test = scaled_dataset_test[:,0:5]

#Outputs
y1_test = scaled_dataset_test[:,5]   #y1 - Adsorption Rate

df_scaled_test = pd.DataFrame({'T': scaled_dataset_test[:, 0], 'Humidity': scaled_dataset_test[:, 1],
                              'AdsTime':scaled_dataset_test[:, 2] , 'DesTime':scaled_dataset_test[:, 3],
                              'SteamVelocity':scaled_dataset_test[:, 4],
                              'q_ads':scaled_dataset_test[:, 5]})

dfin_scaled_test = df_scaled_test[feature_names]
dfout_scaled_test = df_scaled_test[output_names]

print(df_scaled_test)

df_predictions =alm_surr.evaluate_surrogate(dfin_scaled_test)
# Append '_pred' to each column name
df_predictions.rename(columns=lambda x: x + '_pred', inplace=True)
print(df_predictions)

#Put the testing output data next to the predicted data
data= pd.concat([dfout_scaled_test, df_predictions], axis=1)
print(data)

#Performance metrics on the test set
r2_score_test = r2_score(dfout_scaled_test, df_predictions)
print(f"R^2 on testing data : {r2_score_test}")

mse_test = mean_squared_error(dfout_scaled_test, df_predictions)
print('MSE=',mse_test)
rmse_test = np.sqrt(mse_test)
print('RMSE=', rmse_test)

```

```

max_error_test= max_error(dfout_scaled_test, df_predictions)
print('Maximum Absolute Error=', max_error_test)

#Parity plot
z=[0,1]
w=[0,1]
fig, ax= plt.subplots(figsize=(6, 5))
ax.scatter(dfout_scaled_test, df_predictions, alpha=0.3)
ax.plot(dfout_scaled_test, dfout_scaled_test, 'k') #y=x line
ax.set_xlabel('Actual', fontsize=12)
ax.set_ylabel('Predicted', fontsize=12)
ax.set_title('q_ads', fontsize=10)

plt.tight_layout()
plt.savefig(f'alamo_parity_test_q_ads.png', format='png') # Save plot as a PNG file
plt.show()
plt.close(fig) # Close the figure to free memory

```

	T[K]	Humidity (%)	AdsTime[s]	DesTime[s]	SteamVelocity[m/s]	\
0	299.150000	3.555007	4005	1446.223980	0.100000	
1	259.870000	80.000000	2000	1648.294462	0.065586	
2	276.536667	80.000000	5000	1664.918623	0.056617	
3	259.870000	80.000000	3000	828.598523	0.092935	
4	263.000000	80.000000	2000	1900.738003	0.090000	
...	...	...	...	...	...	
4130	259.870000	80.000000	4000	1177.100262	0.049420	
4131	283.837325	42.015591	6000	4940.113688	0.081566	
4132	259.870000	80.000000	6000	3514.737573	0.037825	
4133	255.000000	80.000000	3500	1662.738965	0.072381	
4134	293.000000	73.577187	2400	1150.375983	0.067619	

	q_ads[mol/kg]	q_des[mol/kg]
0	0.897938	0.111569
1	0.714757	0.157852
2	1.333691	0.185728
3	1.067524	0.280458
4	0.557096	0.073018
...	...	...
4130	1.339310	0.327063
4131	1.325406	0.001032
4132	1.361432	0.085728
4133	0.940162	0.136550
4134	0.576758	0.249646

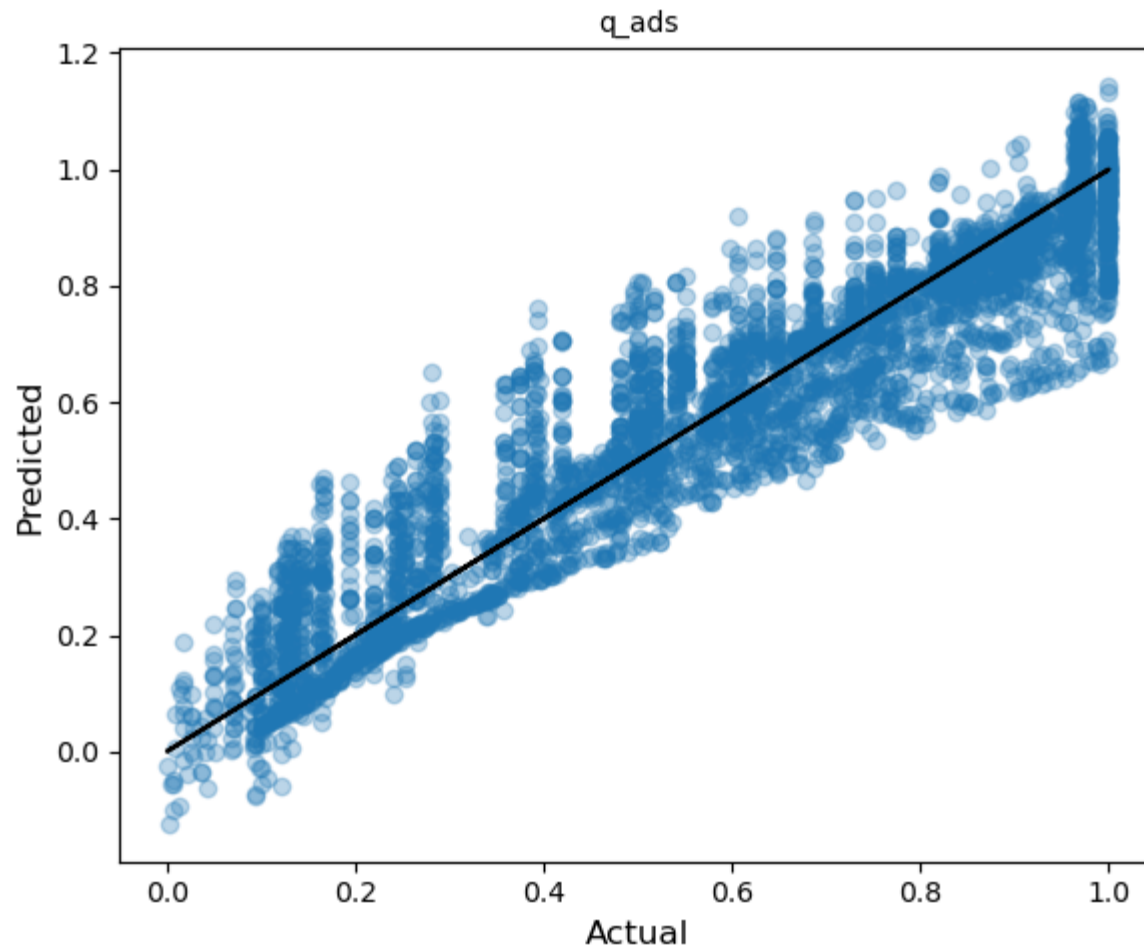
```
[4135 rows x 7 columns]
      T Humidity  AdsTime  DesTime  SteamVelocity  q_ads
0    1.000000  0.000000  0.557159  0.254022    1.000000  0.506675
1    0.110306  1.000000  0.112098  0.301437    0.617624  0.311970
2    0.487807  1.000000  0.778024  0.305338    0.517965  0.969842
3    0.110306  1.000000  0.334073  0.109099    0.921499  0.686930
4    0.181200  1.000000  0.112098  0.360672    0.888889  0.144391
...      ...      ...      ...      ...      ...
4130 0.110306  1.000000  0.556049  0.190873    0.438004  0.975813
4131 0.653167  0.503114  1.000000  1.073849    0.795181  0.961035
4132 0.110306  1.000000  1.000000  0.739390    0.309169  0.999328
4133 0.000000  1.000000  0.445061  0.304826    0.693120  0.551555
4134 0.860702  0.915981  0.200888  0.184602    0.640210  0.165290
```

```
[4135 rows x 6 columns]
      q_ads_pred
0    0.563390
1    0.242720
2    0.848630
3    0.601431
4    0.153815
...      ...
4130 0.748954
4131 0.836116
4132 1.002954
4133 0.651635
4134 0.261390
```

```
[4135 rows x 1 columns]
      q_ads  q_ads_pred
0    0.506675  0.563390
1    0.311970  0.242720
2    0.969842  0.848630
3    0.686930  0.601431
4    0.144391  0.153815
...      ...      ...
4130 0.975813  0.748954
4131 0.961035  0.836116
4132 0.999328  1.002954
4133 0.551555  0.651635
4134 0.165290  0.261390
```

```
[4135 rows x 2 columns]
```

R^2 on testing data : 0.8831318347955406
MSE= 0.010249377816517554
RMSE= 0.10123921086475118
Maximum Absolute Error= 0.3723391774532633



ALAMO Desorption

In [4]: %reset

Once deleted, variables cannot be recovered. Proceed (y/[n])? y

```
In [5]: from idaes.core.surrogate.alamopy import AlamoTrainer, AlamoSurrogate
from idaes.core.surrogate.plotting.sm_plotter import surrogate_scatter2D, surrogate_parity, surrogate_residual
import time
import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import r2_score, mean_squared_error, max_error

df= pd.read_csv('train_data_winsorized.csv')
#print(df)
dataset = np.array(df)

#Scale the data between 0 and 1
scaler = MinMaxScaler((0,1))
print(scaler.fit(dataset))
scaled_dataset = scaler.transform(dataset)
#print(scaled_dataset)

#Inputs, X1 ..... X8 (matrix X)
x = scaled_dataset[:,0:5]

#Outputs
y1 = scaled_dataset[:,6]  #y1 - Desorption Rate

feature_names = ['T', 'Humidity', 'AdsTime', 'DesTime', 'SteamVelocity']
output_names = ['q_des'] #, 'q_ads[mol/kg]'

df_scaled = pd.DataFrame({'T': scaled_dataset[:, 0], 'Humidity': scaled_dataset[:, 1],
                          'AdsTime':scaled_dataset[:, 2] , 'DesTime':scaled_dataset[:, 3],
                          'SteamVelocity':scaled_dataset[:, 4],
                          'q_des':scaled_dataset[:, 6]})

dfin_scaled = df_scaled[feature_names]
dfout_scaled = df_scaled[output_names]

print(df_scaled)
```

```
MinMaxScaler()  
      T Humidity  AdsTime  DesTime  SteamVelocity  q_des  
0      0.653167  1.000000  0.556049  0.697732      0.045917  0.469209  
1      0.110306  1.000000  1.000000  0.218832      0.427028  0.395570  
2      0.940640  0.427037  0.556049  0.382874      0.456776  0.203751  
3      0.000000  1.000000  0.445061  0.203217      0.846560  0.234399  
4      0.339751  1.000000  0.557159  0.000000      0.444444  0.996628  
...      ...      ...      ...      ...      ...  
16533  0.623476  0.553551  0.556049  0.029486      0.451578  0.804262  
16534  0.110306  1.000000  0.112098  0.252773      0.827541  0.183549  
16535  0.634202  0.664214  0.112098  0.360672      0.764549  0.113292  
16536  0.940640  1.000000  0.778024  0.122920      1.000000  0.311489  
16537  0.000000  1.000000  0.223085  0.508044      0.693120  0.061235  
  
[16538 rows x 6 columns]
```

```
In [6]: trainer = AlamoTrainer(input_labels= dfin_scaled,
                             output_labels= dfout_scaled,
                             training_dataframe= df_scaled)
trainer.config.constant = True
trainer.config.linfcns = True
trainer.config.monomialpower = [2, 3]
trainer.config.multi2power = [1, 2]

# Train surrogate (calls ALAMO through IDAES ALAMOPy wrapper)
from time import time
t0 = time()
success, alm_surr, msg = trainer.train_surrogate()
training_time = time() - t0
print(f"Training time = {(training_time)}s")

# save model to JSON
model = alm_surr.save_to_file(f'alamo_pce3_surrogate_des.json', overwrite=True)

# create callable surrogate object
surrogate_expressions = trainer._results['Model']

input_labels = trainer._input_labels
output_labels = trainer._output_labels
#xmin, xmax = [0.1, 0.8], [0.8, 1.2]
#input_bounds = {input_labels[i]: (xmin[i], xmax[i])
                 #for i in range(len(input_labels))}

alm_surr = AlamoSurrogate(surrogate_expressions,
                          input_labels,
                          output_labels)
                          #input_bounds)

surrogate_parity(alm_surr, df_scaled, filename= f'train_parity_alamo_pce3_des.pdf')

df_predictions_train =alm_surr.evaluate_surrogate(dfin_scaled)
# Append '_pred' to each column name
df_predictions_train.rename(columns=lambda x: x + '_pred', inplace=True)
print(df_predictions_train)

#Put the testing output data next to the predicted data
data_train= pd.concat([dfout_scaled, df_predictions_train], axis=1)
print(data_train)
```

```

#Performance metrics on the test set
r2_score_train = r2_score(dfout_scaled, df_predictions_train)
print(f"R^2 on training data : {r2_score_train}")

mse = mean_squared_error(dfout_scaled, df_predictions_train)
print('MSE=',mse)
rmse = np.sqrt(mse)
print('RMSE=', rmse)

max_error_train= max_error(dfout_scaled, df_predictions_train)
print('Maximum Absolute Error=', max_error_train)

#Parity plot
z=[0,1]
w=[0,1]
fig, ax= plt.subplots(figsize=(6, 5))
ax.scatter(dfout_scaled, df_predictions_train, alpha=0.3)
ax.plot(z, w, 'k--') #y=x line
ax.set_xlabel('Actual', fontsize=12)
ax.set_ylabel('Predicted', fontsize=12)
ax.set_title('q_des', fontsize=10)

plt.tight_layout()
plt.savefig(f'alamo_parity_train_des.png', format='png') # Save plot as a PNG file
plt.show()
plt.close(fig) # Close the figure to free memory

```

ALAMO version 2023.2.13. Built: WIN-64 Mon Feb 13 21:30:56 EST 2023

If you use this software, please cite:
 Cozad, A., N. V. Sahinidis and D. C. Miller,
 Automatic Learning of Algebraic Models for Optimization,
 AIChE Journal, 60, 2211-2227, 2014.

ALAMO is powered by the BARON software from <http://www.minlp.com/>

Licensee: Dimitrios Fardis at Georgia Institute of Technology, dfardis3@gatech.edu.

Reading input data
 Checking input consistency and initializing data structures

Step 0: Initializing data set

User provided an initial data set of 16538 data points

We will sample no more data points at this stage

Iteration 1 (Approx. elapsed time 0.73 s)

Step 1: Model building using BIC

Model building for variable q_des

BIC = -0.469E+05 with q_des = 0.33

BIC = -0.629E+05 with q_des = - 1.2 * DesTime*SteamVelocity + 0.53

BIC = -0.780E+05 with q_des = - 2.9 * DesTime*SteamVelocity + 2.9 * (DesTime*SteamVelocity)^2 + 0.68

BIC = -0.110E+06 with q_des = 0.24E-01 * T - 0.17E-01 * Humidity + 0.12E-01 * AdsTime - 3.2 * DesTime - 2.4 * Steam Velocity - 0.82E-01 * T^2 + 0.13E-01 * Humidity^2 + 0.51E-01 * AdsTime^2 + 4.2 * DesTime^2 + 2.8 * SteamVelocity^2 + 0.56E-01 * T^3 - 0.49E-01 * AdsTime^3 - 2.1 * DesTime^3 - 1.3 * SteamVelocity^3 - 0.33E-01 * T*AdsTime + 0.19E-01 * T*DesTime - 0.92E-01 * AdsTime*DesTime + 0.21E-01 * AdsTime*SteamVelocity + 0.25 * DesTime*SteamVelocity + 0.12 * (AdsTime*DesTime)^2 + 0.25 * (DesTime*SteamVelocity)^2 + 1.5

Calculating quality metrics on observed data set.

Quality metrics for output q_des

SSE OLR:	21.5
SSE:	21.6
RMSE:	0.361E-01
R2:	0.978
R2 adjusted:	0.978
Model size:	22
BIC:	-0.110E+06
Cp:	-0.165E+05
AICc:	-0.110E+06
HQC:	-0.110E+06
MSE:	0.131E-02
SSEp:	21.6
RIC:	179.
MADp:	0.411E+04

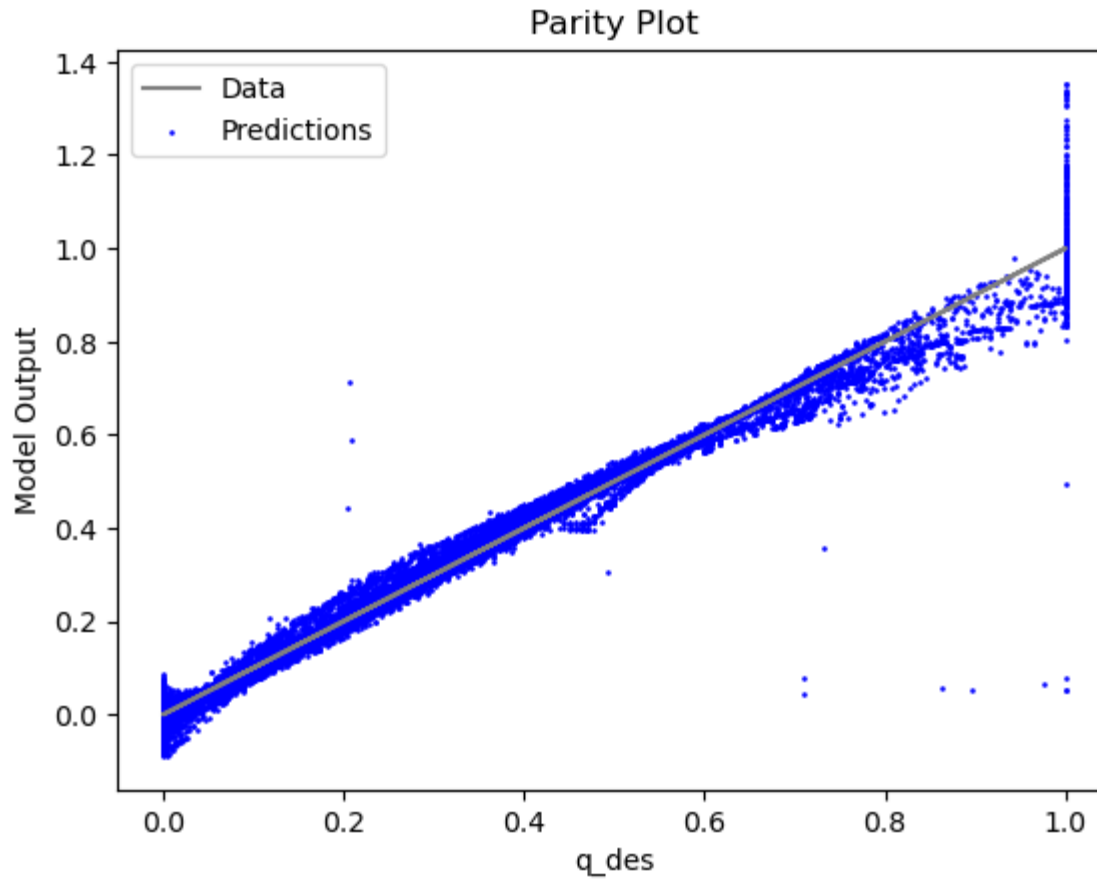
Total execution time 16. s

Times breakdown

OLR time: 12. s in 7880 ordinary linear regression problem(s)
MINLP time: 0.0 s in 0 optimization problem(s)
Simulation time: 0.0 s to simulate 0 point(s)
All other time: 4.6 s in 1 iteration(s)

Normal termination

Training time = 16.82500457763672s



	q_des_pred
0	0.498338
1	0.396173
2	0.179908
3	0.265001
4	0.882905
...	...
16533	0.779659
16534	0.197051
16535	0.107488
16536	0.322268
16537	0.054719

[16538 rows x 1 columns]

	q_des	q_des_pred
0	0.469209	0.498338
1	0.395570	0.396173
2	0.203751	0.179908
3	0.234399	0.265001
4	0.996628	0.882905
...	...	...
16533	0.804262	0.779659
16534	0.183549	0.197051
16535	0.113292	0.107488
16536	0.311489	0.322268
16537	0.061235	0.054719

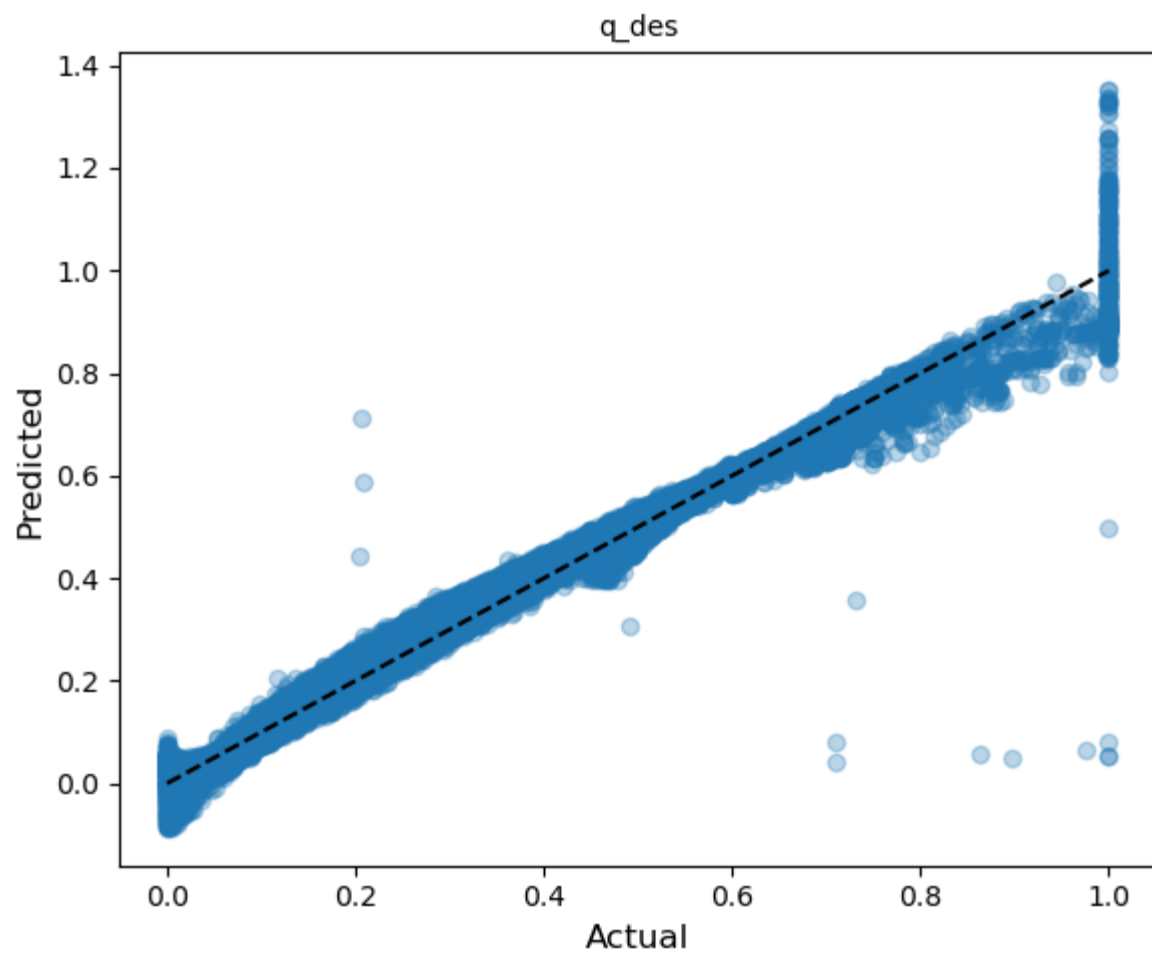
[16538 rows x 2 columns]

R^2 on training data : 0.9777819564069133

MSE= 0.0013050429662266707

RMSE= 0.036125378423300575

Maximum Absolute Error= 0.9478430320203608



```
In [7]: #-----Testing set-----
df_test= pd.read_csv('test_data_unprocessed.csv')
print(df_test)
dataset_test = np.array(df_test)

#Scale the data between 0 and 1 using the scaler based on the training set
scaled_dataset_test = scaler.transform(dataset_test)
#print(scaled_dataset_test)

#Inputs, X1 ..... X8 (matrix X)
x_test = scaled_dataset_test[:,0:5]

#Outputs
y1_test = scaled_dataset_test[:,6]    #y1 - Desorption Rate

df_scaled_test = pd.DataFrame({'T': scaled_dataset_test[:, 0], 'Humidity': scaled_dataset_test[:, 1],
                              'AdsTime':scaled_dataset_test[:, 2] , 'DesTime':scaled_dataset_test[:, 3],
                              'SteamVelocity':scaled_dataset_test[:, 4],
                              'q_des':scaled_dataset_test[:, 6]})

dfin_scaled_test = df_scaled_test[feature_names]
dfout_scaled_test = df_scaled_test[output_names]

print(dfout_scaled_test)

df_predictions =alm_surr.evaluate_surrogate(dfin_scaled_test)
# Append '_pred' to each column name
df_predictions.rename(columns=lambda x: x + '_pred', inplace=True)
print(df_predictions)

#Put the testing output data next to the predicted data
data= pd.concat([dfout_scaled_test, df_predictions], axis=1)
print(data)

#Performance metrics on the test set
r2_score_test = r2_score(dfout_scaled_test, df_predictions)
print(f"R^2 on testing data : {r2_score_test}")

mse_test = mean_squared_error(dfout_scaled_test, df_predictions)
print('MSE=',mse_test)
rmse_test = np.sqrt(mse_test)
print('RMSE=', rmse_test)
```

```

max_error_test= max_error(dfout_scaled_test, df_predictions)
print('Maximum Absolute Error=', max_error_test)

#Parity plot
z=[0,1]
w=[0,1]
fig, ax= plt.subplots(figsize=(6, 5))
ax.scatter(dfout_scaled_test, df_predictions, alpha=0.3)
ax.plot(dfout_scaled_test, dfout_scaled_test, 'k') #y=x line
ax.set_xlabel('Actual', fontsize=12)
ax.set_ylabel('Predicted', fontsize=12)
ax.set_title('q_des', fontsize=10)

plt.tight_layout()
plt.savefig(f'alamo_parity_test_q_des.png', format='png') # Save plot as a PNG file
plt.show()
plt.close(fig) # Close the figure to free memory

```

	T[K]	Humidity (%)	AdsTime[s]	DesTime[s]	SteamVelocity[m/s]	\
0	299.150000	3.555007	4005	1446.223980	0.100000	
1	259.870000	80.000000	2000	1648.294462	0.065586	
2	276.536667	80.000000	5000	1664.918623	0.056617	
3	259.870000	80.000000	3000	828.598523	0.092935	
4	263.000000	80.000000	2000	1900.738003	0.090000	
...	...	...	...	...	...	
4130	259.870000	80.000000	4000	1177.100262	0.049420	
4131	283.837325	42.015591	6000	4940.113688	0.081566	
4132	259.870000	80.000000	6000	3514.737573	0.037825	
4133	255.000000	80.000000	3500	1662.738965	0.072381	
4134	293.000000	73.577187	2400	1150.375983	0.067619	

	q_ads[mol/kg]	q_des[mol/kg]
0	0.897938	0.111569
1	0.714757	0.157852
2	1.333691	0.185728
3	1.067524	0.280458
4	0.557096	0.073018
...	...	...
4130	1.339310	0.327063
4131	1.325406	0.001032
4132	1.361432	0.085728
4133	0.940162	0.136550
4134	0.576758	0.249646

```
[4135 rows x 7 columns]
```

```
      q_des
0      0.141276
1      0.203229
2      0.240543
3      0.367346
4      0.089672
...
4130  0.429731
4131 -0.006687
4132  0.106686
4133  0.174715
4134  0.326102
```

```
[4135 rows x 1 columns]
```

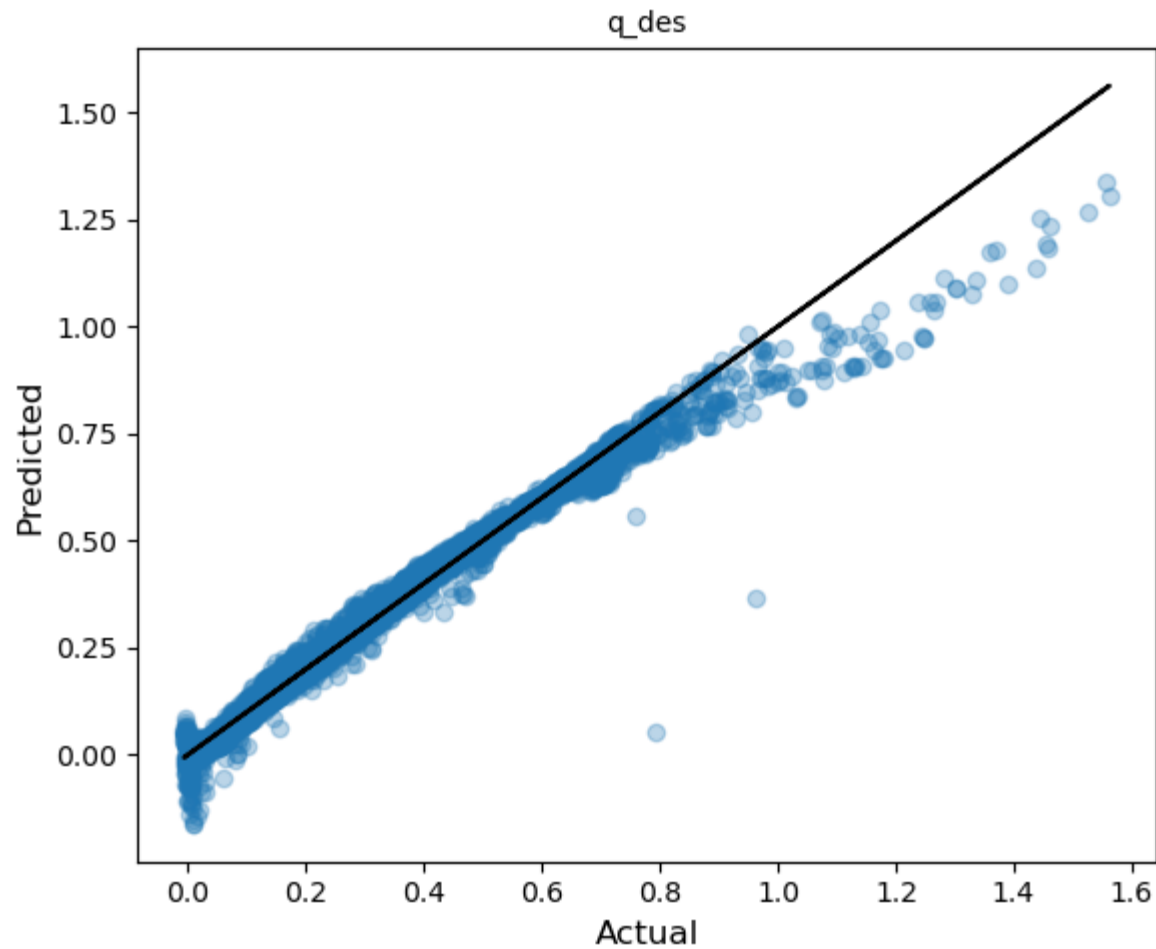
```
      q_des_pred
0      0.129993
1      0.199266
2      0.228576
3      0.401257
4      0.083221
...
4130  0.437829
4131 -0.031274
4132  0.155498
4133  0.180390
4134  0.340235
```

```
[4135 rows x 1 columns]
```

```
      q_des  q_des_pred
0      0.141276  0.129993
1      0.203229  0.199266
2      0.240543  0.228576
3      0.367346  0.401257
4      0.089672  0.083221
...
4130  0.429731  0.437829
4131 -0.006687 -0.031274
4132  0.106686  0.155498
4133  0.174715  0.180390
4134  0.326102  0.340235
```

```
[4135 rows x 2 columns]
```

R² on testing data : 0.9749562670414449
MSE= 0.0015486576284721354
RMSE= 0.03935298754188982
Maximum Absolute Error= 0.739552286273403



Advanced Neural Network

In [1]: %reset

```
In [2]: import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, r2_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow import keras
from sklearn.model_selection import KFold, cross_val_score
from scikeras.wrappers import KerasRegressor
from keras.callbacks import EarlyStopping
import matplotlib.pyplot as plt
import time
```

Adsorption

```
In [3]: train_df = pd.read_csv('train_data_winsorized.csv')
train_dataset = np.array(train_df)

# Normalize data
scaler = MinMaxScaler((0, 1))
scaled_train_dataset = scaler.fit_transform(train_dataset)

# Inputs, X1 ..... X8 (matrix X)
X_train = scaled_train_dataset[:, 0:5]
# Output
y_train = scaled_train_dataset[:, 5] #y1 - Adsorption Rate

def ANN_sigmoid_model_ads():
    model = Sequential()
    model.add(Dense(64, input_dim=X_train.shape[1], activation='sigmoid'))
    model.add(Dense(32, activation='sigmoid'))
    model.add(Dense(1, activation='linear'))
    model.compile(loss='mean_squared_error', optimizer='adam')
    return model
```

```
In [4]: # ANN Model
model = ANN_sigmoid_model_ads()

start_time = time.time()

model.fit(X_train, y_train,
          epochs=100,
          validation_split=0.2, # Use 20% of training data for validation
          verbose=1)

end_time = time.time()
training_time = end_time - start_time
print(f'Time taken to train the model: {training_time:.2f} seconds')

test_df = pd.read_csv('test_data_unprocessed.csv')
test_dataset = np.array(test_df)
scaled_test_dataset = scaler.transform(test_dataset)

X_test = scaled_test_dataset[:, 0:5]
y_test = scaled_test_dataset[:, 5] #y1 - Adsorption Rate

y_pred = model.predict(X_test)
```

C:\Users\khoppner\AppData\Local\anaconda3\envs\school\Lib\site-packages\keras\src\layers\core\dense.py:87: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Epoch 1/100
414/414  2s 2ms/step - loss: 0.2923 - val_loss: 0.0655
Epoch 2/100
414/414  1s 2ms/step - loss: 0.0579 - val_loss: 0.0298
Epoch 3/100
414/414  1s 2ms/step - loss: 0.0263 - val_loss: 0.0181
Epoch 4/100
414/414  1s 2ms/step - loss: 0.0194 - val_loss: 0.0175
Epoch 5/100
414/414  1s 2ms/step - loss: 0.0187 - val_loss: 0.0172
Epoch 6/100
414/414  1s 1ms/step - loss: 0.0186 - val_loss: 0.0172
Epoch 7/100
414/414  1s 1ms/step - loss: 0.0186 - val_loss: 0.0176
Epoch 8/100
414/414  1s 1ms/step - loss: 0.0182 - val_loss: 0.0169
Epoch 9/100
414/414  1s 1ms/step - loss: 0.0180 - val_loss: 0.0166
Epoch 10/100
414/414  1s 1ms/step - loss: 0.0183 - val_loss: 0.0161
Epoch 11/100
414/414  1s 1ms/step - loss: 0.0171 - val_loss: 0.0155
Epoch 12/100
414/414  1s 1ms/step - loss: 0.0169 - val_loss: 0.0163
Epoch 13/100
414/414  1s 1ms/step - loss: 0.0167 - val_loss: 0.0151
Epoch 14/100
414/414  1s 1ms/step - loss: 0.0160 - val_loss: 0.0146
Epoch 15/100
414/414  1s 1ms/step - loss: 0.0162 - val_loss: 0.0146
Epoch 16/100
414/414  1s 2ms/step - loss: 0.0161 - val_loss: 0.0154
Epoch 17/100
414/414  1s 2ms/step - loss: 0.0160 - val_loss: 0.0150
Epoch 18/100
414/414  1s 1ms/step - loss: 0.0156 - val_loss: 0.0145
Epoch 19/100
414/414  1s 2ms/step - loss: 0.0158 - val_loss: 0.0145
Epoch 20/100
414/414  1s 2ms/step - loss: 0.0156 - val_loss: 0.0146
Epoch 21/100
414/414  1s 2ms/step - loss: 0.0157 - val_loss: 0.0146
Epoch 22/100
414/414  1s 1ms/step - loss: 0.0160 - val_loss: 0.0173
```

```
Epoch 23/100
414/414  1s 1ms/step - loss: 0.0160 - val_loss: 0.0146
Epoch 24/100
414/414  0s 1ms/step - loss: 0.0153 - val_loss: 0.0143
Epoch 25/100
414/414  1s 1ms/step - loss: 0.0158 - val_loss: 0.0143
Epoch 26/100
414/414  0s 1ms/step - loss: 0.0156 - val_loss: 0.0142
Epoch 27/100
414/414  1s 1ms/step - loss: 0.0154 - val_loss: 0.0142
Epoch 28/100
414/414  1s 2ms/step - loss: 0.0155 - val_loss: 0.0143
Epoch 29/100
414/414  1s 2ms/step - loss: 0.0156 - val_loss: 0.0159
Epoch 30/100
414/414  1s 2ms/step - loss: 0.0158 - val_loss: 0.0146
Epoch 31/100
414/414  1s 2ms/step - loss: 0.0154 - val_loss: 0.0156
Epoch 32/100
414/414  1s 1ms/step - loss: 0.0159 - val_loss: 0.0140
Epoch 33/100
414/414  1s 1ms/step - loss: 0.0154 - val_loss: 0.0142
Epoch 34/100
414/414  1s 1ms/step - loss: 0.0158 - val_loss: 0.0140
Epoch 35/100
414/414  1s 1ms/step - loss: 0.0153 - val_loss: 0.0144
Epoch 36/100
414/414  1s 1ms/step - loss: 0.0149 - val_loss: 0.0140
Epoch 37/100
414/414  1s 1ms/step - loss: 0.0153 - val_loss: 0.0145
Epoch 38/100
414/414  1s 2ms/step - loss: 0.0151 - val_loss: 0.0138
Epoch 39/100
414/414  1s 1ms/step - loss: 0.0151 - val_loss: 0.0144
Epoch 40/100
414/414  1s 2ms/step - loss: 0.0154 - val_loss: 0.0138
Epoch 41/100
414/414  1s 2ms/step - loss: 0.0149 - val_loss: 0.0137
Epoch 42/100
414/414  1s 2ms/step - loss: 0.0151 - val_loss: 0.0139
Epoch 43/100
414/414  1s 2ms/step - loss: 0.0149 - val_loss: 0.0137
Epoch 44/100
414/414  1s 2ms/step - loss: 0.0147 - val_loss: 0.0136
```

```
Epoch 45/100
414/414  1s 1ms/step - loss: 0.0150 - val_loss: 0.0142
Epoch 46/100
414/414  1s 1ms/step - loss: 0.0148 - val_loss: 0.0140
Epoch 47/100
414/414  1s 1ms/step - loss: 0.0148 - val_loss: 0.0136
Epoch 48/100
414/414  1s 1ms/step - loss: 0.0145 - val_loss: 0.0134
Epoch 49/100
414/414  1s 1ms/step - loss: 0.0147 - val_loss: 0.0134
Epoch 50/100
414/414  1s 1ms/step - loss: 0.0146 - val_loss: 0.0138
Epoch 51/100
414/414  1s 1ms/step - loss: 0.0149 - val_loss: 0.0139
Epoch 52/100
414/414  1s 2ms/step - loss: 0.0146 - val_loss: 0.0132
Epoch 53/100
414/414  1s 2ms/step - loss: 0.0144 - val_loss: 0.0145
Epoch 54/100
414/414  1s 2ms/step - loss: 0.0145 - val_loss: 0.0133
Epoch 55/100
414/414  1s 2ms/step - loss: 0.0144 - val_loss: 0.0133
Epoch 56/100
414/414  1s 2ms/step - loss: 0.0146 - val_loss: 0.0140
Epoch 57/100
414/414  1s 2ms/step - loss: 0.0144 - val_loss: 0.0132
Epoch 58/100
414/414  1s 2ms/step - loss: 0.0144 - val_loss: 0.0134
Epoch 59/100
414/414  1s 2ms/step - loss: 0.0142 - val_loss: 0.0132
Epoch 60/100
414/414  1s 2ms/step - loss: 0.0144 - val_loss: 0.0129
Epoch 61/100
414/414  1s 2ms/step - loss: 0.0141 - val_loss: 0.0139
Epoch 62/100
414/414  1s 2ms/step - loss: 0.0144 - val_loss: 0.0127
Epoch 63/100
414/414  1s 2ms/step - loss: 0.0138 - val_loss: 0.0128
Epoch 64/100
414/414  1s 1ms/step - loss: 0.0136 - val_loss: 0.0127
Epoch 65/100
414/414  0s 1ms/step - loss: 0.0135 - val_loss: 0.0125
Epoch 66/100
414/414  1s 1ms/step - loss: 0.0134 - val_loss: 0.0124
```

```
Epoch 67/100
414/414  1s 2ms/step - loss: 0.0131 - val_loss: 0.0119
Epoch 68/100
414/414  1s 1ms/step - loss: 0.0130 - val_loss: 0.0122
Epoch 69/100
414/414  1s 1ms/step - loss: 0.0124 - val_loss: 0.0121
Epoch 70/100
414/414  1s 1ms/step - loss: 0.0124 - val_loss: 0.0121
Epoch 71/100
414/414  1s 1ms/step - loss: 0.0126 - val_loss: 0.0113
Epoch 72/100
414/414  1s 1ms/step - loss: 0.0122 - val_loss: 0.0118
Epoch 73/100
414/414  1s 1ms/step - loss: 0.0117 - val_loss: 0.0112
Epoch 74/100
414/414  1s 2ms/step - loss: 0.0123 - val_loss: 0.0116
Epoch 75/100
414/414  1s 1ms/step - loss: 0.0122 - val_loss: 0.0111
Epoch 76/100
414/414  1s 1ms/step - loss: 0.0119 - val_loss: 0.0110
Epoch 77/100
414/414  1s 2ms/step - loss: 0.0119 - val_loss: 0.0112
Epoch 78/100
414/414  1s 1ms/step - loss: 0.0120 - val_loss: 0.0110
Epoch 79/100
414/414  1s 2ms/step - loss: 0.0118 - val_loss: 0.0113
Epoch 80/100
414/414  1s 2ms/step - loss: 0.0120 - val_loss: 0.0108
Epoch 81/100
414/414  1s 2ms/step - loss: 0.0118 - val_loss: 0.0108
Epoch 82/100
414/414  1s 1ms/step - loss: 0.0115 - val_loss: 0.0107
Epoch 83/100
414/414  1s 2ms/step - loss: 0.0116 - val_loss: 0.0107
Epoch 84/100
414/414  1s 2ms/step - loss: 0.0112 - val_loss: 0.0109
Epoch 85/100
414/414  1s 2ms/step - loss: 0.0113 - val_loss: 0.0104
Epoch 86/100
414/414  1s 2ms/step - loss: 0.0111 - val_loss: 0.0104
Epoch 87/100
414/414  1s 2ms/step - loss: 0.0113 - val_loss: 0.0103
Epoch 88/100
414/414  1s 2ms/step - loss: 0.0110 - val_loss: 0.0106
```

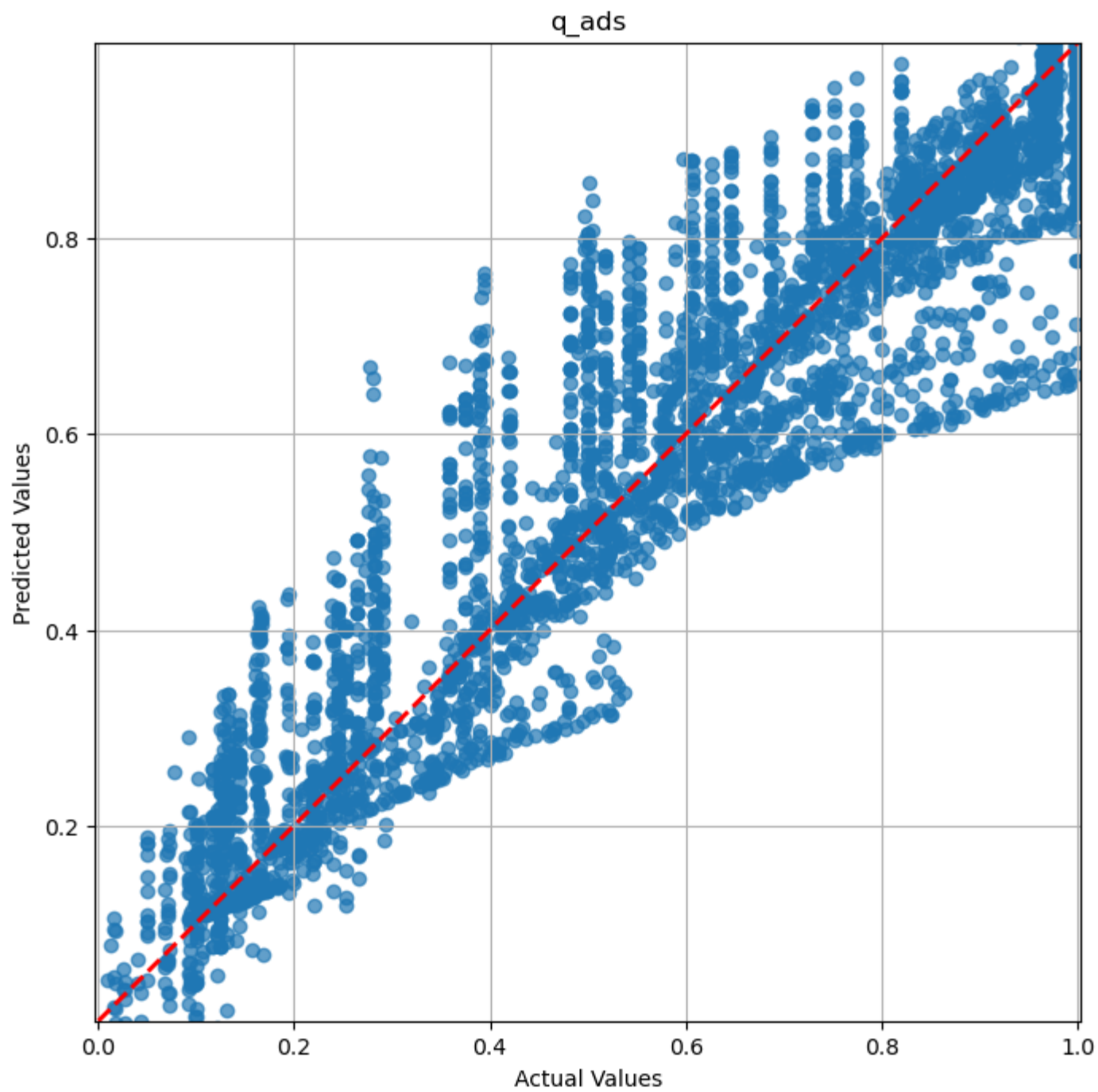
```
Epoch 89/100
414/414 ————— 1s 2ms/step - loss: 0.0109 - val_loss: 0.0102
Epoch 90/100
414/414 ————— 1s 2ms/step - loss: 0.0106 - val_loss: 0.0102
Epoch 91/100
414/414 ————— 1s 2ms/step - loss: 0.0108 - val_loss: 0.0101
Epoch 92/100
414/414 ————— 1s 2ms/step - loss: 0.0108 - val_loss: 0.0100
Epoch 93/100
414/414 ————— 1s 2ms/step - loss: 0.0105 - val_loss: 0.0101
Epoch 94/100
414/414 ————— 1s 2ms/step - loss: 0.0107 - val_loss: 0.0102
Epoch 95/100
414/414 ————— 1s 2ms/step - loss: 0.0106 - val_loss: 0.0103
Epoch 96/100
414/414 ————— 1s 2ms/step - loss: 0.0102 - val_loss: 0.0098
Epoch 97/100
414/414 ————— 1s 2ms/step - loss: 0.0102 - val_loss: 0.0096
Epoch 98/100
414/414 ————— 1s 2ms/step - loss: 0.0102 - val_loss: 0.0097
Epoch 99/100
414/414 ————— 1s 2ms/step - loss: 0.0104 - val_loss: 0.0096
Epoch 100/100
414/414 ————— 1s 2ms/step - loss: 0.0101 - val_loss: 0.0096
Time taken to train the model: 72.00 seconds
130/130 ————— 0s 1ms/step
```

```
In [5]: mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f'Test R2 score: {r2}')
print(f'Test Mean Squared Error: {mse}')
print(f'Test Root Mean Squared Error (RMSE): {rmse}')
```

```
Test R2 score: 0.8845127460970998
Test Mean Squared Error: 0.010128271425944734
Test Root Mean Squared Error (RMSE): 0.10063931352083406
```

```
In [6]: plt.figure(figsize=(8, 8))
plt.scatter(y_test, y_pred, alpha=0.7)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2) # Reference line
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('q_ads')
plt.grid(True)
plt.axis('equal')
plt.xlim([y_test.min(), y_test.max()])
plt.ylim([y_test.min(), y_test.max()])
plt.show()
```



Desorption

```
In [7]: train_df = pd.read_csv('train_data_winsorized.csv')
train_dataset = np.array(train_df)

# Normalize data
scaler = MinMaxScaler((0, 1))
scaled_train_dataset = scaler.fit_transform(train_dataset)

# Inputs, X1 ..... X8 (matrix X)
X_train = scaled_train_dataset[:, 0:5]
# Output
y_train = scaled_train_dataset[:, 6] #y2 - Desorption Rate

def ANN_sigmoid_model_des():
    model = Sequential()
    model.add(Dense(64, input_dim=X_train.shape[1], activation='sigmoid'))
    model.add(Dense(32, activation='sigmoid'))
    model.add(Dense(1, activation='linear'))
    model.compile(loss='mean_squared_error', optimizer='adam')
    return model
```

```
In [8]: # ANN Model
model = ANN_sigmoid_model_des()

start_time = time.time()

model.fit(X_train, y_train,
          epochs=100,
          validation_split=0.2, # Use 20% of training data for validation
          verbose=1)

end_time = time.time()
training_time = end_time - start_time
print(f'Time taken to train the model: {training_time:.2f} seconds')

test_df = pd.read_csv('test_data_unprocessed.csv')
test_dataset = np.array(test_df)
scaled_test_dataset = scaler.transform(test_dataset)

X_test = scaled_test_dataset[:, 0:5]
y_test = scaled_test_dataset[:, 6] #y2 - Desorption Rate

y_pred = model.predict(X_test)
```

Epoch 1/100

C:\Users\khoppner\AppData\Local\anaconda3\envs\school\Lib\site-packages\keras\src\layers\core\dense.py:87: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
414/414  2s 2ms/step - loss: 0.0791 - val_loss: 0.0301
Epoch 2/100
414/414  1s 2ms/step - loss: 0.0234 - val_loss: 0.0158
Epoch 3/100
414/414  1s 2ms/step - loss: 0.0155 - val_loss: 0.0148
Epoch 4/100
414/414  1s 2ms/step - loss: 0.0159 - val_loss: 0.0153
Epoch 5/100
414/414  1s 1ms/step - loss: 0.0156 - val_loss: 0.0143
Epoch 6/100
414/414  1s 2ms/step - loss: 0.0153 - val_loss: 0.0141
Epoch 7/100
414/414  1s 2ms/step - loss: 0.0150 - val_loss: 0.0146
Epoch 8/100
414/414  1s 1ms/step - loss: 0.0147 - val_loss: 0.0136
Epoch 9/100
414/414  1s 1ms/step - loss: 0.0149 - val_loss: 0.0135
Epoch 10/100
414/414  0s 1ms/step - loss: 0.0133 - val_loss: 0.0128
Epoch 11/100
414/414  1s 1ms/step - loss: 0.0134 - val_loss: 0.0130
Epoch 12/100
414/414  1s 1ms/step - loss: 0.0133 - val_loss: 0.0129
Epoch 13/100
414/414  1s 1ms/step - loss: 0.0122 - val_loss: 0.0112
Epoch 14/100
414/414  1s 2ms/step - loss: 0.0119 - val_loss: 0.0102
Epoch 15/100
414/414  1s 2ms/step - loss: 0.0111 - val_loss: 0.0095
Epoch 16/100
414/414  1s 2ms/step - loss: 0.0097 - val_loss: 0.0080
Epoch 17/100
414/414  1s 2ms/step - loss: 0.0086 - val_loss: 0.0068
Epoch 18/100
414/414  1s 2ms/step - loss: 0.0075 - val_loss: 0.0061
Epoch 19/100
414/414  1s 2ms/step - loss: 0.0065 - val_loss: 0.0053
Epoch 20/100
414/414  1s 2ms/step - loss: 0.0059 - val_loss: 0.0051
Epoch 21/100
414/414  1s 2ms/step - loss: 0.0052 - val_loss: 0.0042
Epoch 22/100
414/414  1s 2ms/step - loss: 0.0045 - val_loss: 0.0043
Epoch 23/100
```

```
414/414  1s 2ms/step - loss: 0.0042 - val_loss: 0.0033
Epoch 24/100
414/414  1s 2ms/step - loss: 0.0033 - val_loss: 0.0027
Epoch 25/100
414/414  1s 2ms/step - loss: 0.0030 - val_loss: 0.0026
Epoch 26/100
414/414  1s 1ms/step - loss: 0.0027 - val_loss: 0.0022
Epoch 27/100
414/414  1s 2ms/step - loss: 0.0026 - val_loss: 0.0021
Epoch 28/100
414/414  1s 2ms/step - loss: 0.0022 - val_loss: 0.0017
Epoch 29/100
414/414  1s 2ms/step - loss: 0.0023 - val_loss: 0.0016
Epoch 30/100
414/414  1s 1ms/step - loss: 0.0019 - val_loss: 0.0015
Epoch 31/100
414/414  1s 1ms/step - loss: 0.0016 - val_loss: 0.0014
Epoch 32/100
414/414  1s 1ms/step - loss: 0.0018 - val_loss: 0.0014
Epoch 33/100
414/414  1s 1ms/step - loss: 0.0019 - val_loss: 0.0012
Epoch 34/100
414/414  1s 1ms/step - loss: 0.0015 - val_loss: 0.0017
Epoch 35/100
414/414  1s 1ms/step - loss: 0.0013 - val_loss: 0.0010
Epoch 36/100
414/414  1s 1ms/step - loss: 0.0014 - val_loss: 0.0014
Epoch 37/100
414/414  1s 2ms/step - loss: 0.0012 - val_loss: 9.2078e-04
Epoch 38/100
414/414  1s 1ms/step - loss: 0.0012 - val_loss: 9.7249e-04
Epoch 39/100
414/414  1s 2ms/step - loss: 9.3429e-04 - val_loss: 9.5391e-04
Epoch 40/100
414/414  1s 2ms/step - loss: 0.0011 - val_loss: 9.2948e-04
Epoch 41/100
414/414  1s 2ms/step - loss: 8.9094e-04 - val_loss: 7.3628e-04
Epoch 42/100
414/414  1s 2ms/step - loss: 9.0980e-04 - val_loss: 6.8829e-04
Epoch 43/100
414/414  1s 2ms/step - loss: 6.4645e-04 - val_loss: 6.3708e-04
Epoch 44/100
414/414  1s 2ms/step - loss: 0.0013 - val_loss: 6.2876e-04
Epoch 45/100
```

414/414  1s 2ms/step - loss: 0.0012 - val_loss: 6.1125e-04
Epoch 46/100

414/414  1s 2ms/step - loss: 8.5971e-04 - val_loss: 5.6053e-04
Epoch 47/100

414/414  1s 2ms/step - loss: 5.7875e-04 - val_loss: 5.4122e-04
Epoch 48/100

414/414  1s 2ms/step - loss: 6.9041e-04 - val_loss: 5.1172e-04
Epoch 49/100

414/414  1s 1ms/step - loss: 5.9743e-04 - val_loss: 5.0179e-04
Epoch 50/100

414/414  1s 2ms/step - loss: 9.0544e-04 - val_loss: 4.9257e-04
Epoch 51/100

414/414  1s 2ms/step - loss: 9.0555e-04 - val_loss: 5.1023e-04
Epoch 52/100

414/414  1s 2ms/step - loss: 5.3160e-04 - val_loss: 4.7275e-04
Epoch 53/100

414/414  1s 2ms/step - loss: 9.6466e-04 - val_loss: 5.5428e-04
Epoch 54/100

414/414  1s 1ms/step - loss: 8.3703e-04 - val_loss: 4.7266e-04
Epoch 55/100

414/414  1s 1ms/step - loss: 6.4120e-04 - val_loss: 7.8223e-04
Epoch 56/100

414/414  1s 1ms/step - loss: 8.1998e-04 - val_loss: 5.3333e-04
Epoch 57/100

414/414  1s 1ms/step - loss: 6.7368e-04 - val_loss: 4.5299e-04
Epoch 58/100

414/414  1s 1ms/step - loss: 7.8239e-04 - val_loss: 5.3875e-04
Epoch 59/100

414/414  1s 1ms/step - loss: 7.7465e-04 - val_loss: 4.5230e-04
Epoch 60/100

414/414  1s 2ms/step - loss: 6.5197e-04 - val_loss: 4.3308e-04
Epoch 61/100

414/414  1s 2ms/step - loss: 6.4345e-04 - val_loss: 4.3408e-04
Epoch 62/100

414/414  1s 2ms/step - loss: 9.1792e-04 - val_loss: 5.2245e-04
Epoch 63/100

414/414  1s 2ms/step - loss: 5.4584e-04 - val_loss: 4.2853e-04
Epoch 64/100

414/414  1s 2ms/step - loss: 5.0107e-04 - val_loss: 4.3641e-04
Epoch 65/100

414/414  1s 2ms/step - loss: 5.8789e-04 - val_loss: 4.2699e-04
Epoch 66/100

414/414  1s 2ms/step - loss: 6.2815e-04 - val_loss: 4.3293e-04
Epoch 67/100

414/414  1s 1ms/step - loss: 4.0707e-04 - val_loss: 5.4036e-04
Epoch 68/100

414/414  1s 2ms/step - loss: 6.4348e-04 - val_loss: 4.4106e-04
Epoch 69/100

414/414  1s 2ms/step - loss: 5.7494e-04 - val_loss: 4.0880e-04
Epoch 70/100

414/414  1s 1ms/step - loss: 6.8515e-04 - val_loss: 4.0890e-04
Epoch 71/100

414/414  1s 2ms/step - loss: 9.6450e-04 - val_loss: 4.1821e-04
Epoch 72/100

414/414  1s 2ms/step - loss: 8.8740e-04 - val_loss: 4.0976e-04
Epoch 73/100

414/414  1s 2ms/step - loss: 4.2693e-04 - val_loss: 4.7802e-04
Epoch 74/100

414/414  1s 2ms/step - loss: 6.2693e-04 - val_loss: 4.0878e-04
Epoch 75/100

414/414  1s 1ms/step - loss: 0.0010 - val_loss: 4.2329e-04
Epoch 76/100

414/414  1s 2ms/step - loss: 6.6724e-04 - val_loss: 3.9924e-04
Epoch 77/100

414/414  1s 1ms/step - loss: 7.3756e-04 - val_loss: 4.0116e-04
Epoch 78/100

414/414  1s 1ms/step - loss: 7.3740e-04 - val_loss: 4.0390e-04
Epoch 79/100

414/414  1s 1ms/step - loss: 6.5830e-04 - val_loss: 3.9243e-04
Epoch 80/100

414/414  1s 1ms/step - loss: 6.0771e-04 - val_loss: 4.9177e-04
Epoch 81/100

414/414  1s 1ms/step - loss: 5.9269e-04 - val_loss: 4.6145e-04
Epoch 82/100

414/414  1s 1ms/step - loss: 3.8984e-04 - val_loss: 4.8545e-04
Epoch 83/100

414/414  1s 1ms/step - loss: 4.9263e-04 - val_loss: 4.3429e-04
Epoch 84/100

414/414  1s 2ms/step - loss: 6.7289e-04 - val_loss: 3.8102e-04
Epoch 85/100

414/414  1s 1ms/step - loss: 6.3872e-04 - val_loss: 4.2905e-04
Epoch 86/100

414/414  1s 2ms/step - loss: 5.1347e-04 - val_loss: 3.7004e-04
Epoch 87/100

414/414  1s 2ms/step - loss: 4.1645e-04 - val_loss: 3.6781e-04
Epoch 88/100

414/414  1s 2ms/step - loss: 3.6161e-04 - val_loss: 4.2112e-04
Epoch 89/100

```

414/414 ————— 1s 2ms/step - loss: 5.1223e-04 - val_loss: 3.8819e-04
Epoch 90/100
414/414 ————— 1s 2ms/step - loss: 3.8938e-04 - val_loss: 3.6904e-04
Epoch 91/100
414/414 ————— 1s 2ms/step - loss: 6.0164e-04 - val_loss: 3.7345e-04
Epoch 92/100
414/414 ————— 1s 2ms/step - loss: 4.7895e-04 - val_loss: 4.2461e-04
Epoch 93/100
414/414 ————— 1s 2ms/step - loss: 6.2489e-04 - val_loss: 3.9212e-04
Epoch 94/100
414/414 ————— 1s 2ms/step - loss: 7.1947e-04 - val_loss: 3.7284e-04
Epoch 95/100
414/414 ————— 1s 2ms/step - loss: 6.8271e-04 - val_loss: 3.7662e-04
Epoch 96/100
414/414 ————— 1s 2ms/step - loss: 5.6387e-04 - val_loss: 4.1220e-04
Epoch 97/100
414/414 ————— 1s 2ms/step - loss: 9.8440e-04 - val_loss: 3.8854e-04
Epoch 98/100
414/414 ————— 1s 1ms/step - loss: 8.2689e-04 - val_loss: 3.6513e-04
Epoch 99/100
414/414 ————— 1s 2ms/step - loss: 4.8525e-04 - val_loss: 4.4325e-04
Epoch 100/100
414/414 ————— 1s 1ms/step - loss: 5.2563e-04 - val_loss: 3.9663e-04
Time taken to train the model: 71.09 seconds
130/130 ————— 0s 1ms/step

```

```

In [9]: mse = mean_squared_error(y_test, y_pred)
        rmse = np.sqrt(mse)
        r2 = r2_score(y_test, y_pred)

        print(f'Test R² score: {r2}')
        print(f'Test Mean Squared Error: {mse}')
        print(f'Test Root Mean Squared Error (RMSE): {rmse}')

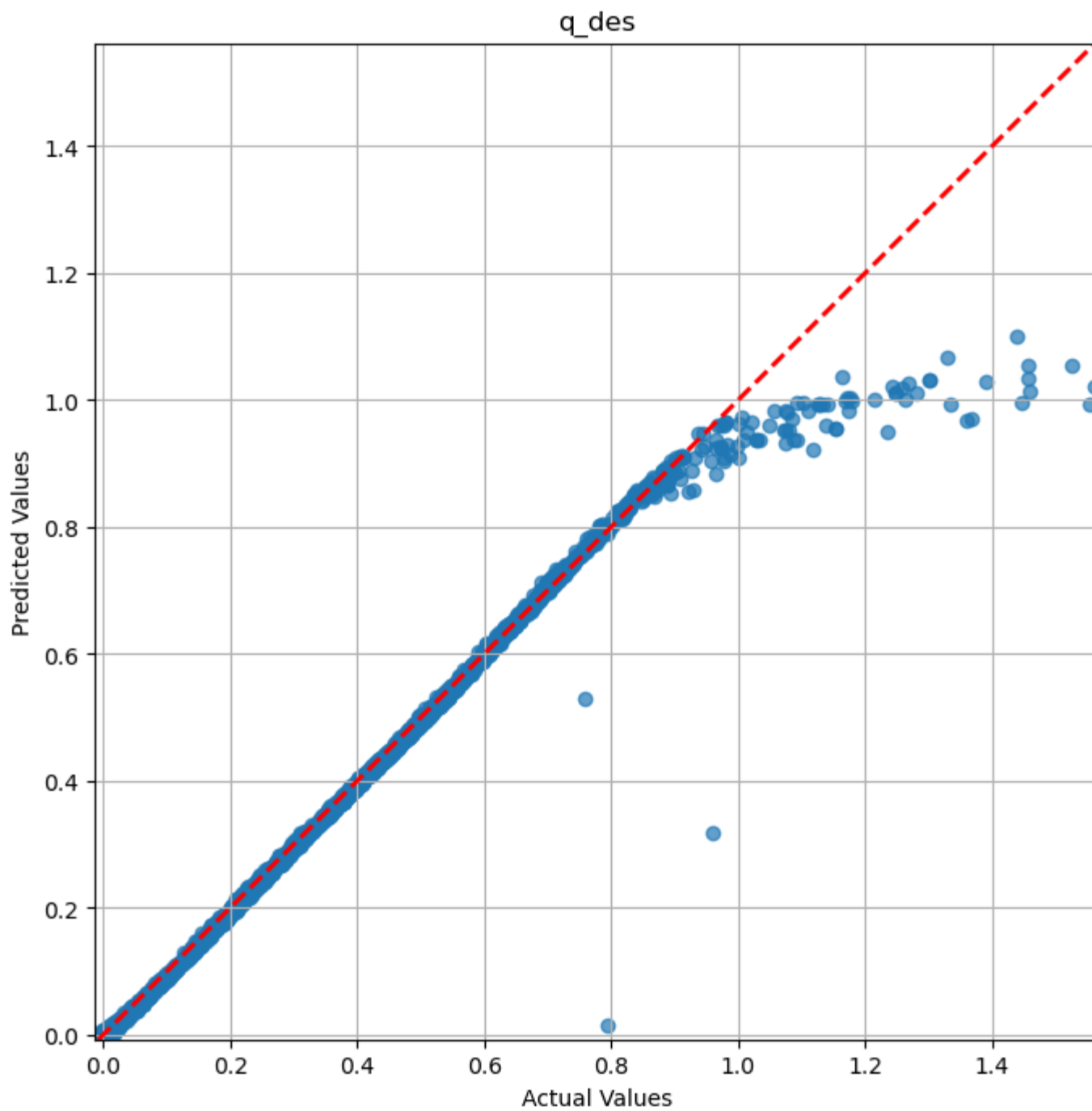
```

```

Test R² score: 0.9803481707451355
Test Mean Squared Error: 0.0012152323832610315
Test Root Mean Squared Error (RMSE): 0.03486018335093824

```

```
In [10]: plt.figure(figsize=(8, 8))
plt.scatter(y_test, y_pred, alpha=0.7)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2) # Reference line
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('q_des')
plt.grid(True)
plt.axis('equal')
plt.xlim([y_test.min(), y_test.max()])
plt.ylim([y_test.min(), y_test.max()])
plt.show()
```



Kriging

In [11]: `%reset`

```
In [24]: import numpy as np
import time

from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

```
In [13]: # Load the data
training_data_path = r"train_data_winsorized.csv"
testing_data_path = r"test_data_unprocessed.csv"

headers = [ x for x in np.genfromtxt(training_data_path, dtype="str", delimiter=',', max_rows=1) ]

training_data = np.genfromtxt(training_data_path, delimiter=',', skip_header=1)
X_train = training_data[:,5]
ads_train = training_data[:,5].reshape(-1,1)
des_train = training_data[:,6].reshape(-1,1)

testing_data_full = np.genfromtxt(testing_data_path, delimiter=',', skip_header=1)
testing_data, validation_data = train_test_split(testing_data_full, test_size=0.2, random_state=256)
X_test = testing_data[:,5]
X_val = validation_data[:,5]
ads_test = testing_data[:,5].reshape(-1,1)
ads_val = validation_data[:,5].reshape(-1,1)
des_test = testing_data[:,6].reshape(-1,1)
des_val = validation_data[:,6].reshape(-1,1)

print(f"ads_train: {ads_train.shape}")
print(f"des_train: {des_train.shape}")
print()
print(f"ads_test: {ads_test.shape}")
print(f"des_test: {des_test.shape}")
print()
print(f"ads_val: {ads_val.shape}")
print(f"des_val: {des_val.shape}")

ads_train: (16538, 1)
des_train: (16538, 1)

ads_test: (3308, 1)
des_test: (3308, 1)

ads_val: (827, 1)
des_val: (827, 1)
```

```
In [14]: # Shrink datasets for training
idx_small = np.linspace(0, 16537, num=2000, dtype="int")
X_train = X_train[idx_small]
ads_train = ads_train[idx_small]
des_train = des_train[idx_small]

print(f"X_train: {X_train.shape}\nads_train: {ads_train.shape}\ndes_train: {des_train.shape}")
print()
print(f"X_test: {X_test.shape}\nads_test: {ads_test.shape}\ndes_test: {des_test.shape}")
print()
print(f"X_val: {X_val.shape}\nads_val: {ads_val.shape}\ndes_val: {des_val.shape}")

X_train: (2000, 5)
ads_train: (2000, 1)
des_train: (2000, 1)

X_test: (3308, 5)
ads_test: (3308, 1)
des_test: (3308, 1)

X_val: (827, 5)
ads_val: (827, 1)
des_val: (827, 1)
```

```
In [15]: # Scale the data
X_scaler = StandardScaler()
X_scaler.fit(X_train)
X_train_scaled = X_scaler.transform(X_train)
X_val_scaled = X_scaler.transform(X_val)

ads_scaler = StandardScaler()
ads_scaler.fit(ads_train)
ads_train_scaled = ads_scaler.transform(ads_train)
ads_val_scaled = ads_scaler.transform(ads_val)

des_scaler = StandardScaler()
des_scaler.fit(des_train)
des_train_scaled = des_scaler.transform(des_train)
des_val_scaled = des_scaler.transform(des_val)

# No scaling
#X_train_scaled = X_train
#X_test_scaled = X_test
```

```
In [16]: constant_value = 1.0
constant_value_bounds = (1e-3, 1e3)
ck = ConstantKernel(constant_value, constant_value_bounds)

num_features = 5
length_scale_isotropic = 1.0
length_scale_anisotropic = np.ones(num_features)
length_scale_bounds = (1e-2, 1e2)
rbf = RBF(length_scale_isotropic, length_scale_bounds)

krig_kernel = ck * rbf
```

```
In [17]: start_time_1 = time.time()
```

```
In [18]: num_restarts = 25
gpr_ads = GaussianProcessRegressor(kernel=krig_kernel, n_restarts_optimizer=num_restarts)
gpr_ads.fit(X_train_scaled, ads_train_scaled)
params_ads = gpr_ads.get_params()
print(params_ads)
print(gpr_ads.kernel_)
```

C:\Users\khoppner\AppData\Local\anaconda3\envs\school\Lib\site-packages\sklearn\gaussian_process_gpr.py:659: ConvergenceWarning: lbfgs failed to converge (status=2):
ABNORMAL_TERMINATION_IN_LNSRCH.

Increase the number of iterations (max_iter) or scale the data as shown in:

```
https://scikit-learn.org/stable/modules/preprocessing.html
_check_optimize_result("lbfgs", opt_res)
{'alpha': 1e-10, 'copy_X_train': True, 'kernel__k1': 1**2, 'kernel__k2': RBF(length_scale=1), 'kernel__k1__constant_value': 1.0, 'kernel__k1__constant_value_bounds': (0.001, 1000.0), 'kernel__k2__length_scale': 1.0, 'kernel__k2__length_scale_bounds': (0.01, 100.0), 'kernel': 1**2 * RBF(length_scale=1), 'n_restarts_optimizer': 25, 'n_targets': None, 'normalize_y': False, 'optimizer': 'fmin_l_bfgs_b', 'random_state': None}
0.892**2 * RBF(length_scale=0.135)
```

```
In [19]: end_time_1 = time.time()
start_time_2 = time.time()
```

```
In [20]: num_restarts = 25
gpr_des = GaussianProcessRegressor(kernel=krig_kernel, n_restarts_optimizer=num_restarts)
gpr_des.fit(X_train_scaled, des_train_scaled)
params_des = gpr_des.get_params()
print(params_des)
print(gpr_des.kernel_)
```

C:\Users\khoppner\AppData\Local\anaconda3\envs\school\Lib\site-packages\sklearn\gaussian_process_gpr.py:659: ConvergenceWarning: lbfgs failed to converge (status=2):
ABNORMAL_TERMINATION_IN_LNSRCH.

Increase the number of iterations (max_iter) or scale the data as shown in:

```
https://scikit-learn.org/stable/modules/preprocessing.html
_check_optimize_result("lbfgs", opt_res)
{'alpha': 1e-10, 'copy_X_train': True, 'kernel__k1': 1**2, 'kernel__k2': RBF(length_scale=1), 'kernel__k1_constant_value': 1.0, 'kernel__k1_constant_value_bounds': (0.001, 1000.0), 'kernel__k2_length_scale': 1.0, 'kernel__k2_length_scale_bounds': (0.01, 100.0), 'kernel': 1**2 * RBF(length_scale=1), 'n_restarts_optimizer': 25, 'n_targets': None, 'normalize_y': False, 'optimizer': 'fmin_l_bfgs_b', 'random_state': None}
31.2**2 * RBF(length_scale=0.0101)
```

```
In [21]: end_time_2 = time.time()

time_delta_1 = end_time_1 - start_time_1
time_delta_2 = end_time_2 - start_time_2
print(f"time 1 (ads): {time_delta_1}")
print(f"time 2 (des): {time_delta_2}")
```

```
time 1 (ads): 1052.5409243106842
time 2 (des): 903.9253401756287
```

```
In [25]: # make predictions on the validation set
ads_pred_scaled = gpr_ads.predict(X_val_scaled)
des_pred_scaled = gpr_des.predict(X_val_scaled)

ads_pred = ads_scaler.inverse_transform(ads_pred_scaled.reshape(-1,1))
des_pred = des_scaler.inverse_transform(des_pred_scaled.reshape(-1,1))

ads_score = gpr_ads.score(X_val_scaled, ads_val_scaled)
print(f"score on adsorption data: {ads_score}")
ads_mae = mean_absolute_error(ads_pred, ads_val)
print(f"MAE on adsorption data: {ads_mae}")

des_score = gpr_des.score(X_val_scaled, des_val_scaled)
print(f"score on desorption data: {des_score}")
des_mae = mean_absolute_error(des_pred, des_val)
print(f"MAE on desorption data: {des_mae}")

score on adsorption data: 0.5174091020121023
MAE on adsorption data: 0.13953909625162964
score on desorption data: 0.07233549592211874
MAE on desorption data: 0.1364395620046887
```

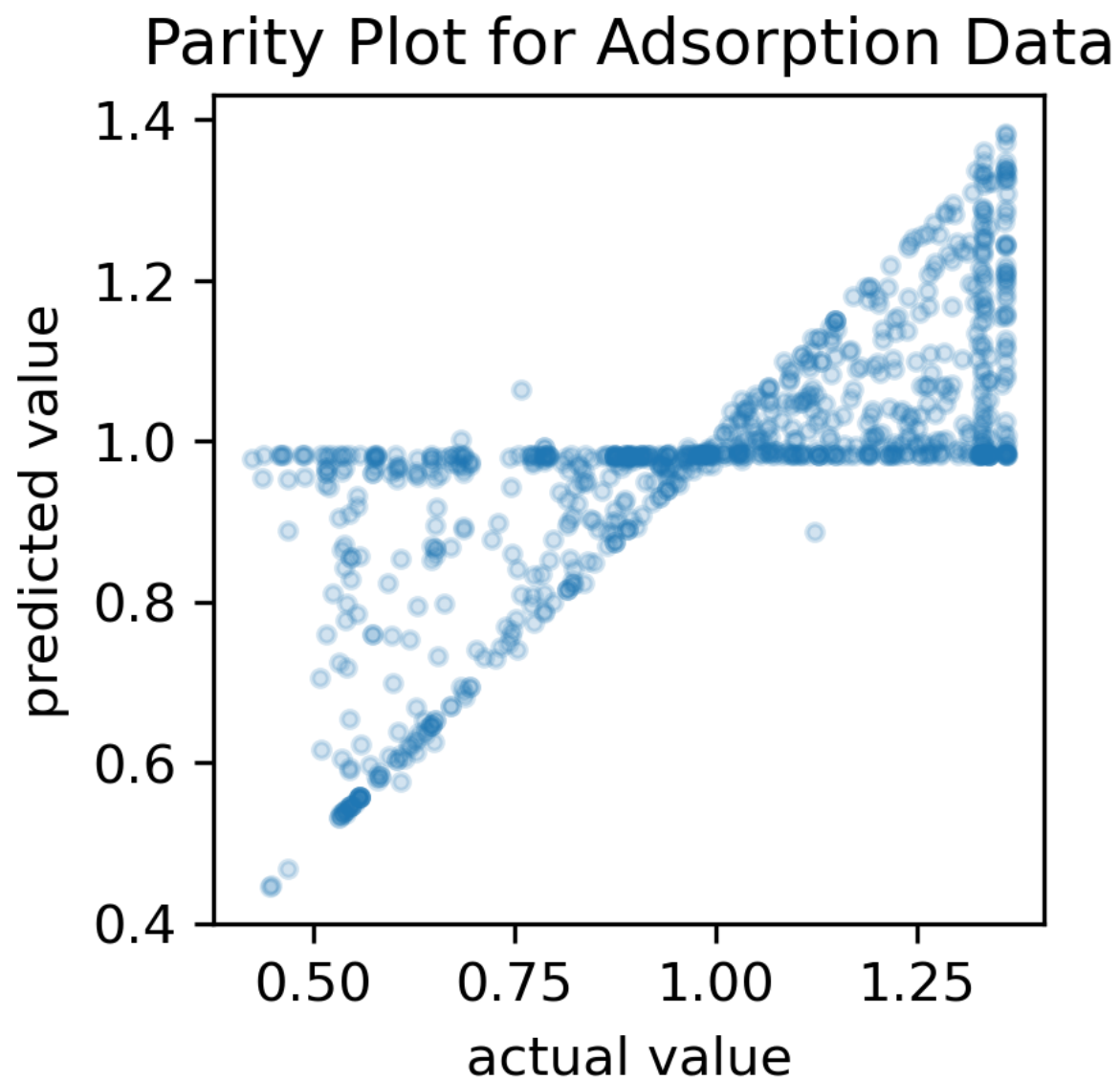
```
In [26]: import matplotlib.pyplot as plt
```

```
In [27]: fig, axes = plt.subplots(2,1, figsize=(7,7), dpi=300)
plt.subplots_adjust(hspace=0.5)

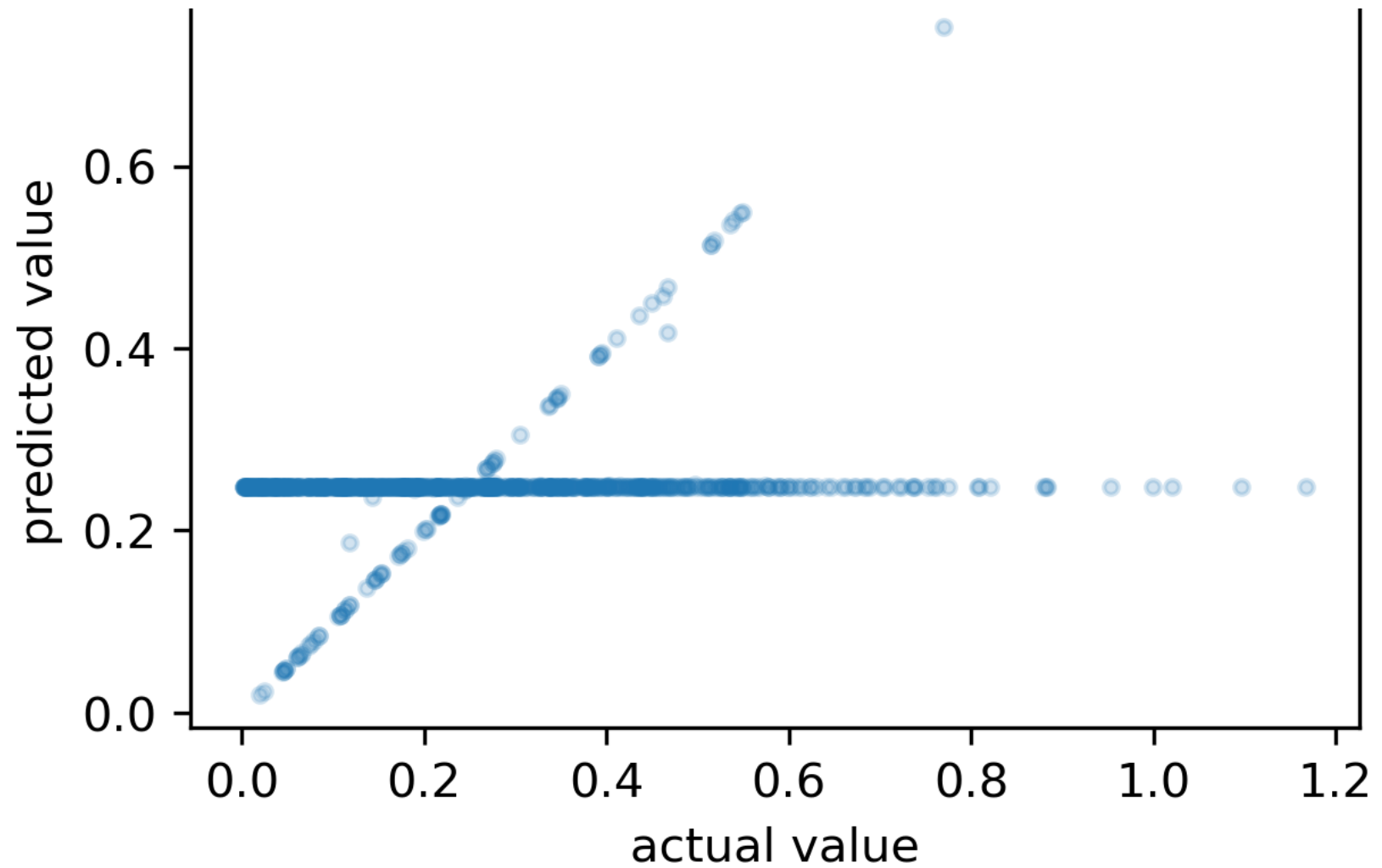
axes[0].scatter(ads_val, ads_pred, marker='.', alpha=0.2)
axes[0].set_aspect("equal")
axes[0].set_title("Parity Plot for Adsorption Data")
axes[0].set_xlabel('actual value')
axes[0].set_ylabel('predicted value')

axes[1].scatter(des_val, des_pred, marker='.', alpha=0.2)
axes[1].set_aspect("equal")
axes[1].set_title("Parity Plot for Desorption Data")
axes[1].set_xlabel('actual value')
axes[1].set_ylabel('predicted value')

plt.show()
```



Parity Plot for Desorption Data



Linear Regression with RBF

In [28]: `%reset`

```
In [29]: import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.metrics import r2_score
import matplotlib.pyplot as plt
from time import time
```

```
In [30]: df1 = pd.read_csv('train_data_winsorized.csv')
df2 = pd.read_csv('test_data_unprocessed.csv')

train_dataset = np.array(df1)
val_dataset = np.array(df2)

#Scale the data between 0 and 1
scaler = MinMaxScaler((0,1))
train_dataset_scaled = scaler.fit_transform(train_dataset)
test_dataset_scaled = scaler.transform(val_dataset)

#Inputs
x_train = train_dataset_scaled[:,0:5]
x_val = test_dataset_scaled[:,0:5]

#Outputs
y1_train = train_dataset_scaled[:,5]    #y1 - Adsorption Rate
y1_val = test_dataset_scaled[:,5]
y2_train = train_dataset_scaled[:,6]    #y2 - Desorption Rate
y2_val = test_dataset_scaled[:,6]

feature_names = ['T[K]', 'Humidity (%)', 'AdsTime[s]', 'DesTime[s]', 'SteamVelocity[m/s]']
output_names = ['q_ads[mol/kg]', 'q_des[mol/kg]']
```

```
In [31]: from sklearn.linear_model import LinearRegression
from sklearn.metrics.pairwise import rbf_kernel
from sklearn.model_selection import train_test_split
import numpy as np

# Sample 20% of the training data (Hold-out)
# Using the rest 80% to optimize the hyperparameter
x_train_sample, x_test_sample, y1_train_sample, y1_test_sample, y2_train_sample, y2_test_sample = train_test_split(x_

print("20% of training set was sampled for the fitting")

gamma_values = [0.1, 0.05, 0.01, 0.005, 0.001]

best_gamma_ads = None
best_score_ads = -100
y1_pred_best = None
best_model_ads = None

start_time = time()
for gamma in gamma_values:
    X_train_rbf = rbf_kernel(x_train_sample, gamma=gamma)
    X_test_rbf = rbf_kernel(x_test_sample, x_train_sample, gamma=gamma)

    model_rbf_ads = LinearRegression()
    model_rbf_ads.fit(X_train_rbf, y1_train_sample)
    y1_pred = model_rbf_ads.predict(X_test_rbf)
    ads_score = r2_score(y1_test_sample, y1_pred)
    if ads_score > best_score_ads:
        best_score_ads = ads_score
        best_gamma_ads = gamma
        y1_pred_best = y1_pred
        best_model_ads = model_rbf_ads

print("<Adsorption Model>")
print(f"LinearRegression using RBF kernel fit done in {(time() - start_time)}s")
print(f"Optimal gamma: {best_gamma_ads} (with score: {best_score_ads})")

best_gamma_des = None
best_score_des = -100
y2_pred_best = None
best_model_des = None

start_time = time()
for gamma in gamma_values:
```

```
gamma = gamma_values
X_train_rbf = rbf_kernel(x_train_sample, gamma=gamma)
X_test_rbf = rbf_kernel(x_test_sample, x_train_sample, gamma=gamma)

model_rbf_des = LinearRegression()
model_rbf_des.fit(X_train_rbf, y2_train_sample)
y2_pred = model_rbf_des.predict(X_test_rbf)
des_score = r2_score(y2_test_sample, y2_pred)

if des_score > best_score_des:
    best_score_des = des_score
    best_gamma_des = gamma
    y2_pred_best = y2_pred
    best_model_des = model_rbf_des

print("<Desorption Model>")
print(f"LinearRegression using RBF kernel fit done in {(time() - start_time)}s")
print(f"Optimal gamma: {best_gamma_des} (with score: {best_score_des})")
```

20% of training set was sampled for the fitting

<Adsorption Model>

LinearRegression using RBF kernel fit done in 51.876747608184814s

Optimal gamma: 0.05 (with score: 0.9526005828101249)

<Desorption Model>

LinearRegression using RBF kernel fit done in 50.075894594192505s

Optimal gamma: 0.05 (with score: 0.9849991430287068)

```

In [32]: #Evaluating r2 score with validation set (unwinsorized test set)
X_val_rbf = rbf_kernel(x_val, x_train_sample, gamma=best_gamma_ads)
y1_val_pred = best_model_ads.predict(X_val_rbf)
r2_ads = best_model_ads.score(X_val_rbf, y1_val) #get the "score", which is equivalent to r^2
print('Validation set r^2 q_ads= {}'.format(r2_ads))

X_val_rbf = rbf_kernel(x_val, x_train_sample, gamma=best_gamma_des)
y2_val_pred = best_model_des.predict(X_val_rbf)
r2_des = best_model_des.score(X_val_rbf, y2_val) #get the "score", which is equivalent to r^2
print('Validation set r^2 q_des= {}'.format(r2_des))

#Parity plot
z=[0,1]
w=[0,1]
fig, axes= plt.subplots(2, figsize=(6, 10))
axes[0].scatter(y1_val, y1_val_pred, alpha=0.3)
axes[0].plot(z, w, 'k--') #y=x line
axes[0].set_xlabel('Actual', fontsize=12)
axes[0].set_ylabel('Predicted', fontsize=12)
axes[0].set_title('q_ads', fontsize=10)

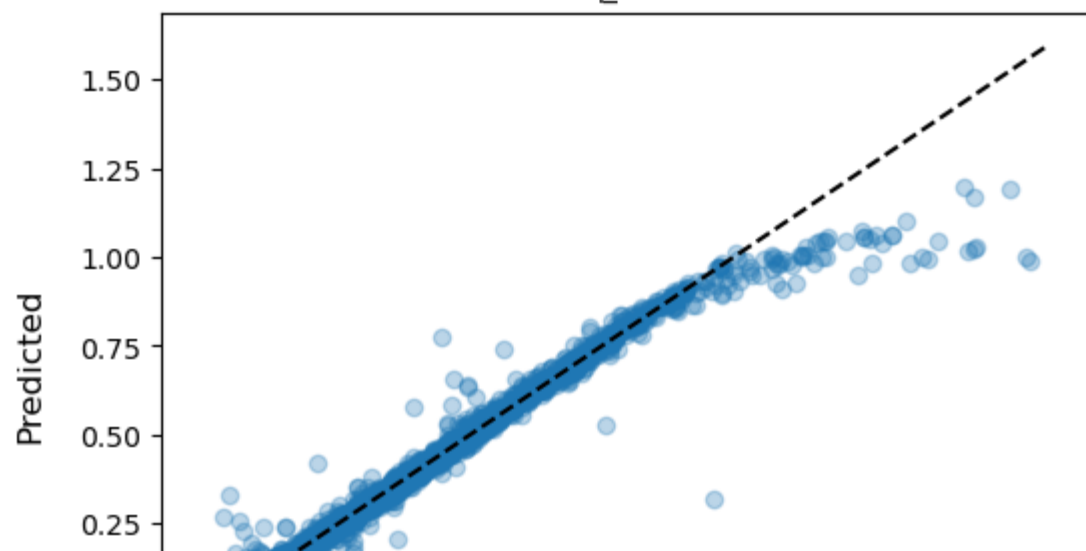
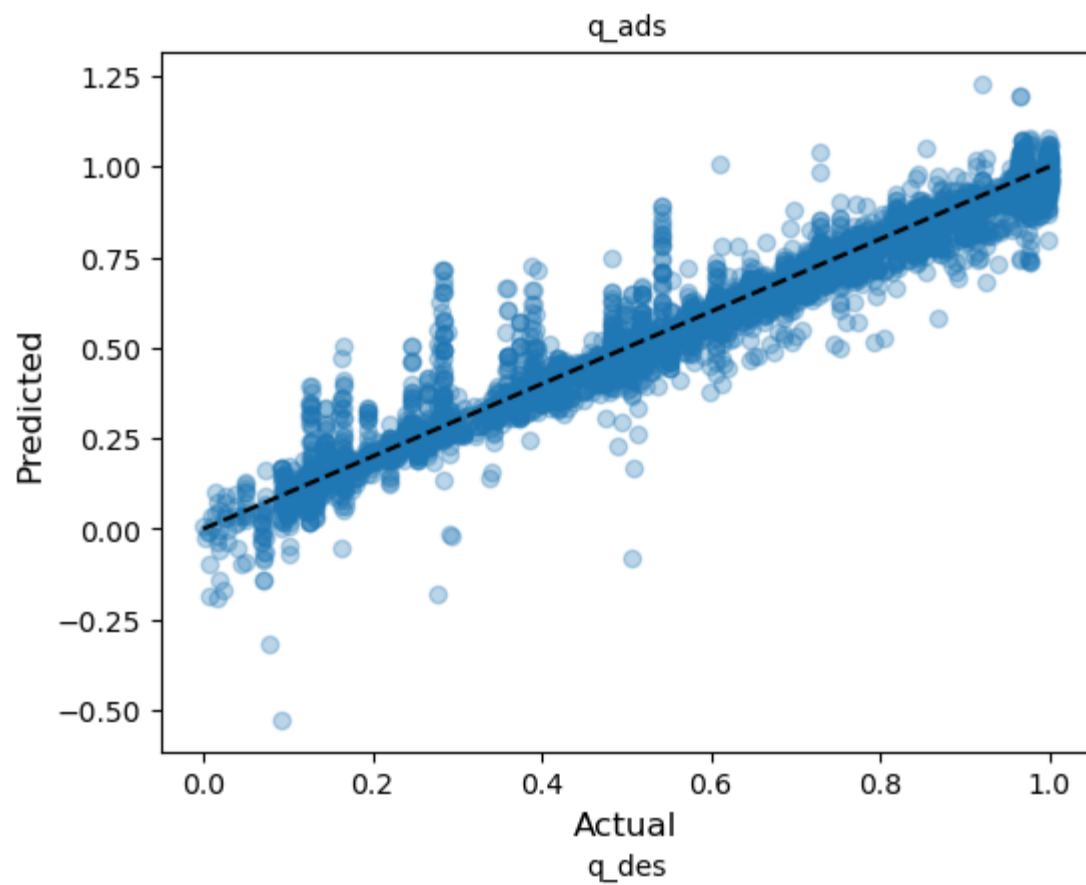
z=[0,1.6]
w=[0,1.6]
axes[1].scatter(y2_val, y2_val_pred, alpha=0.3)
axes[1].plot(z, w, 'k--') #y=x line
axes[1].set_xlabel('Actual', fontsize=12)
axes[1].set_ylabel('Predicted', fontsize=12)
axes[1].set_title('q_des', fontsize=10)

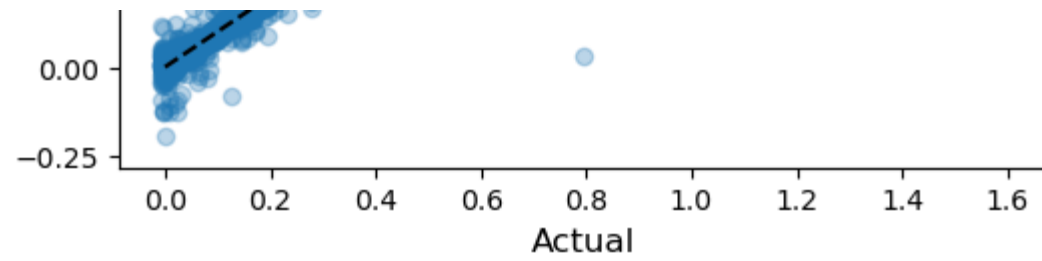
```

Validation set r^2 q_ads= 0.9441421902380225

Validation set r^2 q_des= 0.9767117014371456

Out[32]: Text(0.5, 1.0, 'q_des')





In []: